



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Programmazione Funzionale

Lezioni, Lab e Simulazioni

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

giugno 2026

Indice

1	Introduzione alla Programmazione Funzionale	4
1.1	Spiegazione teorica	4
2	Espressioni e tipi	4
2.1	Introduzione a ML	4
2.2	Espressioni e Comandi	5
2.3	Tipi di dato e astrazione	6
2.4	Espressioni, tipi e operatori in ML	6
2.5	Sistemi di tipi: Sicurezza e Controlli	8
3	Sistemi di tipi, variabili e funzioni	8
3.1	Spiegazione teorica	8
3.2	Implementazione del codice	10
3.2.1	Conversioni di tipo esplicite in ML	10
3.2.2	Ambiente e Shadowing	10
3.2.3	Manipolazione di Liste e Stringhe	10
3.2.4	Dichiarazione e uso di funzioni	11
4	Laboratorio ML 1	11
4.1	Correzione di Type Errors (Esercizi 1.3 - 1.5)	11
4.2	Istanze di Tipi Complessi (Esercizio 1.6 - 1.7)	13
5	Regole di scope	13
5.1	Funzioni e inferenza di tipi	13
5.2	Ambiente (Environment) e Blocchi	14
5.3	Scoping Statico vs Dinamico	14
6	Laboratorio ML 2	15
6.1	Esercizi base sulle Tuple	15
6.2	Esercizi base sulle Liste e sulle Stringhe	16
7	Astrazione del controllo e ricorsione	16
7.1	Spiegazione teorica	16
7.2	Implementazione del codice	17
7.2.1	Mutua Ricorsione in ML	17
8	Laboratorio ML 3	17
8.1	Esercizi base sulla Ricorsione Matematica	18
8.2	Esercizi base sulla Ricorsione e Liste	18
9	Pattern ed environment locale in ML	19
9.1	Spiegazione teorica	19
9.2	Implementazione del codice	20
9.2.1	Merge e uso di as	20
9.2.2	Utilizzo del case e <i>wildcard</i>	20
10	Laboratorio ML 4	21
10.1	Destrutturazione avanzata di liste e tuple	21

10.2 Insiemi Logici tramite Liste (Esercizi L4.6 - L4.8)	22
11 Input-output, eccezioni e funzioni polimorfe	22
11.1 Spiegazione teorica	22
11.2 Implementazione del codice	25
11.2.1 Esempio di I/O e gestione dei flussi di lettura da File	25
11.2.2 Eccezioni personalizzate e meccanismi di recupero (handle)	25
11.2.3 Polimorfismo puro vs Polimorfismo di uguaglianza	26
12 Laboratorio ML 5	26
12.1 Ottimizzazione dell'Ambiente Locale (let val)	26
12.2 Uso delle espressioni case	27
12.3 Esercizi di Input/Output e Stampa	27
13 Funzioni di ordine superiore e curried	28
13.1 Spiegazione teorica	28
13.2 Implementazione del codice	29
13.2.1 Funzioni di Ordine Superiore Classiche (Implementate a mano)	29
13.2.2 Curryng e Applicazione Parziale	30
14 Laboratorio ML 6	30
14.1 Elaborazione di Flussi di Testo (I/O)	30
14.2 Esercizi dal Tutorato: I/O Sicuro ed Eccezioni	31
14.3 Combinare I/O e Ordine Superiore	31
15 Tipi di dati e composizione di funzioni	32
15.1 Spiegazione teorica	32
15.2 Implementazione del codice	33
15.2.1 Composizione e Primitive List	33
15.2.2 Costrutti Datatype e Union Types	33
16 Laboratorio ML 7	34
16.1 Curryng e Composizione Pratica (Esercizio L7.1 - L7.4)	34
16.2 Tipi di dato per gli Alberi Binari (Esercizi L7.5 - L7.10)	34
16.3 Esercizi e approfondimenti dal tutorato: Analisi di Struttura sugli Alberi	35
17 Abstract Data Types in ML e Funtori	36
17.1 Spiegazione teorica	36
17.2 Implementazione del codice: Trasparenza vs Opacità	38
18 Laboratorio ML 8 - Alberi Binari di Ricerca	38
18.1 Creazione di un BST tramite Functor (MakeBST)	39
18.2 Esercizi e approfondimenti dal tutorato (Strutture e BST)	40
19 Lambda Calcolo	41
19.1 Spiegazione teorica	41
19.2 Implementazione e Riduzioni Logiche	42
19.2.1 Esempio 1: Applicazione di Base	42
19.2.2 Esempio 2: Valutazione di un Booleano con operatore	43
19.3 Esercizi e approfondimenti dal tutorato: Computazione in Forma Normale	43

19.3.1	Esercizio L11.12: Riduzione annidata	43
19.3.2	Esercizio L11.13: Riduzione con Catture Evitate	44
20	Programmazione Logica e Prolog	44
20.1	Spiegazione teorica	44
20.2	Implementazione del codice	46
20.2.1	Liste, Ricorsione e Backtracking (append e member)	46
20.2.2	Tic-Tac-Toe in Prolog (Tris Interattivo)	47
21	Simulazioni d'Esame: Preparazione Teorica	48
21.1	Ambiente, Regole di Scope e Passaggio dei Parametri	48
21.2	Memoria, Macchine Astratte e Garbage Collection	50
21.3	Lambda Calcolo e Codifiche	51
21.4	Sistemi di Tipi, Polimorfismo e ML	52
22	Simulazioni d'Esame: Esercizi Pratici in ML	53
22.1	Esercizio 1: Estrazione dei percorsi su Alberi (Appello 15/07/2025)	53
22.2	Esercizio 2: Astrazione del Controllo (Il ciclo FOR)	54
22.3	Esercizio 3: Somma Filtrata per Parità (Prog 1)	54
22.4	Esercizio 4: Manipolazione di stringhe (Prefissi e Suffissi)	55
22.5	Esercizio 5: Addizione Binaria Nativa su Liste	55
22.6	Esercizio 6: Aggregazione su Strutture Ramificate (get_words)	56

1 Introduzione alla Programmazione Funzionale

1.1 Spiegazione teorica

La prima lezione del corso è introduttiva e mira a fornire una panoramica sugli argomenti trattati, le modalità di svolgimento e un inquadramento storico dei linguaggi di programmazione.

Il paradigma funzionale si distingue da quello imperativo e orientato agli oggetti per il suo approccio matematico alla computazione, basato sulla valutazione di espressioni e sull'uso di funzioni pure.

I punti chiave affrontati in questa introduzione includono:

- **Obiettivi del corso:** Comprendere i principi base della programmazione funzionale, studiare il sistema dei tipi, le funzioni di ordine superiore, l'astrazione dei dati e il polimorfismo. Verranno inoltre introdotti il Lambda Calcolo, la programmazione logica (Prolog) e accenni a Scala.
- **Storia dei linguaggi funzionali e logici:**
 - **Anni '50 - '70:** Nascita del LISP (lista processing), capostipite dei linguaggi funzionali (untyped), seguito dalle sue evoluzioni come Scheme e Racket.
 - **Anni '70 - '80:** Sviluppo dei theorem prover (LCF) da cui deriva la famiglia ML (Meta Language), caratterizzata da *call-by-value*, tipizzazione forte e polimorfismo. Nascono Caml e Miranda.
 - **Anni '90 - oggi:** Consolidamento di Standard ML, OCaml e nascita di Haskell (linguaggio funzionale puro *lazy*). Più recentemente, la programmazione funzionale ha influenzato linguaggi moderni come Scala e F#.
 - Sviluppo di strumenti basati sui tipi dipendenti (come Coq) nati negli anni '80.

Trattandosi di una lezione introduttiva sul contesto e sulla storia del paradigma, non sono stati affrontati esercizi pratici o costrutti sintattici del linguaggio ML.

2 Espressioni e tipi

2.1 Introduzione a ML

ML (Meta Language) è un linguaggio di programmazione general-purpose, nato originariamente come metalinguaggio per sistemi di verifica formale (come il theorem prover LCF negli anni '70). Esistono diversi dialetti derivati, tra cui Standard ML, OCaml, F#, ed elementi che hanno influenzato linguaggi come Scala e Haskell.

Perché studiare ML? Studiare la programmazione funzionale impone un cambio di paradigma: i linguaggi mainstream si concentrano sulla gestione degli "stati", mentre la programmazione funzionale si concentra sui **valori**. ML è un linguaggio pratico e compatto le cui idee possono rendere programmatori migliori in qualsiasi linguaggio.

Caratteristiche principali della Programmazione Funzionale in ML:

1. **Linguaggio funzionale:** Il modello di calcolo di base si fonda sulla definizione e sull'applicazione di funzioni. Le funzioni sono trattate come valori (cittadini di prima classe) e possono essere passate come parametri ad altre funzioni, supportando nativamente le *funzioni di ordine superiore* (Higher-order functions).
2. **Ricorsione:** Fortemente incoraggiata rispetto ai cicli `while` (costrutti iterativi sono disponibili, ma secondari).
3. **Programmazione basata su regole:** Costrutti come `if-then-else` sono spesso implementati o sostituiti dal *pattern matching*.
4. **Nessun side effect (effetti collaterali):** La computazione procede per valutazione di espressioni, non tramite l'assegnazione di valori a variabili che mutano lo stato. Se in C `a = b + c` modifica lo stato di `a`, in ML `b + c` crea un nuovo elemento. Gli effetti collaterali (es. stampare a video) sono permessi ma non sono il mezzo principale di calcolo.
5. **Strong typing (Tipizzazione forte):** Tutti i valori e le variabili hanno tipi determinabili a "tempo di compilazione" (*compile-time*). Rileva errori semantici preziosi; inoltre, non è solitamente necessaria la dichiarazione esplicita del tipo grazie alla *Type Inference*.
6. **Polimorfismo:** Una funzione può accettare argomenti di tipi differenti (es. una funzione di ordinamento generica).
7. **Abstract Data Types (Strutture):** Sistema di tipi elegante con la capacità di costruire nuovi tipi complessi.

Eseguire ML: L'interprete interattivo PolyML si avvia da linea di comando digitando `poly` (o `rlwrap poly` in Linux per avere la cronologia dei comandi). All'interno dell'ambiente interattivo (indicato dal prompt `>`), si possono valutare le espressioni chiudendole con il punto e virgola `;`. Per caricare un file esterno si usa `use "foo";`. Per uscire `CTRL+D`, per interrompere `CTRL+C`.

2.2 Espressioni e Comandi

Vi è una netta separazione tra espressioni e comandi:

- **Espressioni:** Entità sintattiche la cui valutazione restituisce un valore o non termina (espressione indefinita). Sono composte da costanti, variabili o operatori applicati a ulteriori espressioni. Notazioni utilizzabili: infissa (`a + b`), prefissa o polacca (`+ a b`), suffissa (`a b +`).
- **Comandi:** Entità sintattiche la cui valutazione non restituisce necessariamente un valore, ma provoca un effetto collaterale (modifica dello stato). Tipici dei linguaggi imperativi, sono del tutto assenti nei paradigmi puramente funzionali o logici.

Strategie di valutazione delle espressioni:

- **Eager evaluation (Valutazione ansiosa):** Si valutano prima tutti gli operandi, poi si applica l'operatore ai risultati.
- **Lazy evaluation (Valutazione pigra):** Gli operandi vengono valutati solo se e quando il loro valore è effettivamente necessario per il calcolo. Questo approccio previene errori in espressioni come `a == 0 ? b : b/a` (se `a` è 0, valutare ansiosamente `b/a` causerebbe una divisione per zero). Viene usata per operatori booleani (short-circuit evaluation), dove il risultato finale può essere noto prima di valutare tutti gli operandi.

In ML si utilizza principalmente la **eager evaluation**, tranne che per alcuni specifici costrutti come `andalso`, `orelse` e `if-then-else` che adottano una valutazione *lazy*.

2.3 Tipi di dato e astrazione

Un **tipo di dato** è una collezione di valori (omogenei) dotata di un insieme di operatori applicabili. I tipi servono a tre scopi principali:

1. **Livello concettuale (Design):** Organizzare l'informazione e documentare il codice (es. distinguere un `prezzo` da un `numero di stanza`).
2. **Livello di programma (Correctness):** Identificare ed evitare errori. I compilatori eseguono il type-checking: assegnare una stringa a un intero (`3 + "pippo"`) solleverà un errore semantico.
3. **Livello implementativo:** Permettere al compilatore ottimizzazioni. Conoscere il tipo permette di sapere quanta memoria allocare (un booleano occupa meno di un double) e calcolare gli offset di memoria per l'accesso ai campi di una struct.

2.4 Espressioni, tipi e operatori in ML

In ML, le espressioni possono essere valori o funzioni applicate a valori. Ogni espressione in PolyML termina con `;`. Esempio:

```
1 + 2 * 3;  
val it = 7: int
```

In questo output: `val` indica il valore dell'espressione, `it` è il nome automatico associato al risultato, e `int` è il tipo dedotto automaticamente.

Tipi Base in ML:

- **int**: Interi positivi e negativi. L'operatore unario per il negativo è la tilde (es. `~3`). Supporta esadecimali (es. `0x124`).
- **real**: Numeri reali. Richiedono il punto decimale o la notazione esponenziale E/e (es. `123.0`, `3.14e12`).
- **bool**: `true` o `false` (rigorosamente minuscoli, ML è case-sensitive).
- **char**: Singolo carattere indicato dal prefisso `#` (es. `#"a"`).
- **string**: Doppi apici (es. `"foo"`). Caratteri speciali supportati: `\n` (newline), `\t` (tab), `\\`, `\"`.
- **unit**: Possiede un unico valore `()`, utilizzato per indicare le espressioni che non restituiscono un valore utile (analogo a `void`).

Operatori:

- **Aritmetici**: `+`, `-`, `*` per `int` e `real`. La divisione per reali è `/`, mentre per gli interi si usano `div` (divisione troncata all'intero inferiore) e `mod` (resto).
- **Forma Infissa/Prefissa**: In ML, un operatore come `+` è di fatto una funzione prefissa che riceve in input una tupla numerica `+(1,2)`, ma consente la notazione infissa per comodità.
- **Stringhe**: La concatenazione avviene tramite l'operatore `^` (es. `"house" ^ "cat" → "housecat"`).
- **Confronto**: `=`, `<`, `>`, `<=`, `>=`, `<>` (diverso). Valgono per interi, caratteri e stringhe (ordine lessicografico). **Attenzione**: in ML non è permesso testare l'uguaglianza (o la disuguaglianza `<>`) tra due valori `real`, per prevenire problemi di approssimazione *floating-point*.
- **Logici**: `not`, `andalso` (AND lazy), `orelse` (OR lazy). `andalso` ha precedenza su `orelse`.

Costrutto If-then-else:

```
if then else ;
```

Poiché in ML tutto è un'espressione, l'utilizzo di `if` richiede **obbligatoriamente** la presenza del ramo `else`. Inoltre, per preservare la *type safety*, e devono avere lo stesso tipo ben definito. Un'espressione come `if 5<6 then 5 else 6.0`; solleva un *Type mismatch error* statico in compilazione.

2.5 Sistemi di tipi: Sicurezza e Controlli

I linguaggi di programmazione adottano strategie diverse per il *Type Checking*:

- **Static Type Checking:** Effettuato a tempo di compilazione (es. C, Java, Haskell, ML) prima dell'esecuzione. Vantaggi: maggiore efficienza a runtime e blocco preventivo di errori. Svantaggi: design del linguaggio più complesso e regole spesso molto rigide.
- **Dynamic Type Checking:** Eseguito durante l'esecuzione del codice (es. Python, JavaScript, Lisp). Vantaggi: massima flessibilità, non servono dichiarazioni di tipo. Svantaggi: gli errori di tipo vengono identificati solo a runtime, e causano perdita di efficienza dovuta ai continui controlli dinamici.

Inoltre, i linguaggi si dividono in:

- **Strongly typed (Tipizzazione forte):** Molto severi sui tipi. Il tipo di un valore non cambia in modo inaspettato. Le conversioni di tipo implicite (coercion) sono vietate (es. ML).
- **Weakly typed (Tipizzazione debole):** Molto flessibili, permettono la conversione implicita tra tipi differenti a runtime (es. JavaScript: `4 + "2" = "42"`, PHP, C/C++).

ML, a garanzia della sua **Type Safety**, è un linguaggio **fortemente e staticamente tipizzato** (statically and strongly typed). Tenta di risolvere l'inconveniente della verbosità grazie al potente meccanismo della *Type Inference* (in cui il compilatore deduce il tipo in automatico), rifiutandosi categoricamente di effettuare coercion nascoste (come `1 + 2.0` che genererà un errore).

3 Sistemi di tipi, variabili e funzioni

3.1 Spiegazione teorica

Il sistema dei tipi di un linguaggio è definito dai suoi tipi predefiniti, dai meccanismi per crearne di nuovi, dalle regole per determinare se il controllo è statico o dinamico e dalle regole di equivalenza, compatibilità e inferenza.

Static e Dynamic Type Checking:

- **Controllo Statico:** Avviene a tempo di compilazione (es. ML, Java, C). Previene l'esecuzione di codice con errori di tipo ed è molto efficiente a runtime, sebbene renda la progettazione e la scrittura del codice più rigida.
- **Controllo Dinamico:** Avviene durante l'esecuzione (es. Python, JavaScript). Garantisce grande flessibilità, ma gli errori vengono scoperti solo a runtime, impattando sulle prestazioni.

I linguaggi si dividono inoltre in **fortemente tipizzati** (strongly typed), dove il tipo di un valore non cambia in modo inaspettato (niente conversioni implicite, come in ML), e **debolmente tipizzati** (weakly typed), dove le conversioni implicite (coercion) sono permesse (es. `4 + "2" = "42"` in JavaScript). ML assicura un alto livello di *Type Safety*, ovvero previene la presenza di errori di tipo inosservati a runtime.

Equivalenza e Compatibilità: Due tipi possono essere considerati equivalenti per **nome** (se hanno letteralmente lo stesso nome) o per **struttura** (se, analizzando le definizioni, la struttura di base è identica). ML utilizza principalmente l'equivalenza strutturale (fatta eccezione per i tipi definiti con **datatype**).

In ML **non esistono conversioni automatiche** (coercion). Se si valuta `1 + 2.0`, il compilatore non converte automaticamente l'intero in reale, ma restituisce un errore di tipo statico. È necessario eseguire una conversione esplicita tramite apposite funzioni di sistema.

Nomi, Variabili e Ambiente (Environment): L'**Environment** è l'insieme delle associazioni tra identificatori (nomi) e valori. In ML, a differenza dei linguaggi imperativi come il C, **le variabili non sono modificabili**. Una variabile è semplicemente un identificatore legato a un valore matematico. Tramite il costrutto **val**, non si assegna un nuovo valore a una locazione di memoria, ma si crea un nuovo legame (binding) nell'ambiente. Se si riutilizza lo stesso nome, la nuova dichiarazione oscura la precedente (*shadowing*), ma il valore originale non viene sovrascritto.

Tipi Complessi: Tuple e Liste:

- **Tuple:** Aggregati di dimensione fissa che possono contenere tipi eterogenei. Esempio: `(4, 5.0, "sei")` è di tipo `int * real * string`. Si accede all'*n*-esimo elemento tramite l'operatore `#n`.
- **Liste:** Aggregati di dimensione variabile, ma rigorosamente omogenei (tutti gli elementi devono avere lo stesso tipo). Esempio: `[1, 2, 3]` è di tipo `int list`.

Funzioni come valori: In ML le funzioni sono trattate come "cittadini di prima classe", ovvero come normali valori. Possono essere anonime tramite il costrutto **fn** (che riflette il lambda calcolo), o associate a un nome con il costrutto **fun**. Una particolarità di ML è che **tutte le funzioni accettano esattamente un solo parametro**. Per simulare più parametri, si passa semplicemente una singola tupla contenente i vari argomenti.

3.2 Implementazione del codice

3.2.1 Conversioni di tipo esplicite in ML

```
> real(4);          (* Da intero a reale *)
val it = 4.0: real

> floor(3.5);       (* Arrotondamento per difetto *)
val it = 3: int

> ceil(3.5);        (* Arrotondamento per eccesso *)
val it = 4: int

> round(3.5);       (* Arrotondamento al pari più vicino *)
val it = 4: int

> trunc(3.5);       (* Troncamento *)
val it = 3: int

> ord("#a");        (* Da char ad ASCII *)
val it = 97: int

> chr(97);          (* Da ASCII a char *)
val it = "#a": char

> str("#a");        (* Da char a string *)
val it = "a": string
```

3.2.2 Ambiente e Shadowing

```
> val pi = 3.14159;
val pi = 3.14159: real

> val pi = 3;
val pi = 3: int
(* Crea un nuovo 'pi' di tipo intero che nasconde il precedente,
   ma non lo sovrascrive fisicamente *)
```

3.2.3 Manipolazione di Liste e Stringhe

```
> val L = [2, 3, 4];
> hd(L);           (* Head: estrae il primo elemento *)
val it = 2 : int

> tl(L);           (* Tail: estrae il resto della lista *)
val it = [3, 4] : int list

> [1, 2] @ [3, 4];  (* Concatenazione di liste, stesso tipo! *)
val it = [1, 2, 3, 4] : int list
```

```

> 1 :: [2, 3];      (* Cons: aggiunge l'elemento in testa alla lista *)
val it = [1, 2, 3] : int list

> explode("abcd"); (* Da stringa a lista di caratteri *)
val it = ["a", "b", "c", "d"] : char list

> implode(["a", "b"]); (* Da lista di caratteri a stringa *)
val it = "ab" : string

```

3.2.4 Dichiarazione e uso di funzioni

```

(* Sintassi della funzione anonima associata a una variabile *)
> val increment = fn n => n + 1;
val increment = fn: int -> int

(* Zucchero sintattico (più comune) *)
> fun increment n = n + 1;
val increment = fn: int -> int

(* Funzione a finti "parametri multipli" (prende in realtà una tupla) *)
> fun max3(a:real, b, c) =
    if a > b then
        if a > c then a else c
    else
        if b > c then b else c;
val max3 = fn: real * real * real -> real

```

4 Laboratorio ML 1

Nel primo laboratorio si mette in pratica la sintassi base affrontata nelle lezioni teoriche, concentrandosi sulla correzione dei "Type Mismatch" e sulla familiarizzazione con i tipi complessi (`list`, `tuple` e relative combinazioni).

4.1 Correzione di Type Errors (Esercizi 1.3 - 1.5)

ML è un linguaggio in cui la correttezza del tipo (*Type Safety*) viene imposta in modo rigoroso. Analizziamo come correggere espressioni sintatticamente o logicamente invalide.

```

(* ERRORE: ceil accetta solo real, non int *)
ceil(4);
(* CORREZIONE *)
ceil(4.0);

(* ERRORE: i rami dell'if devono restituire esattamente lo stesso tipo *)
if true then 5 + 6 else 7.0;
(* CORREZIONE (converti l'intero in reale) *)
if true then real(5 + 6) else 7.0;

(* ERRORE: la condizione dell'if deve essere strettamente di tipo bool *)
if 0 then 1 else 2;
(* CORREZIONE *)
if false then 1 else 2;

(* ERRORE: ord si applica ai caratteri (prefisso #), non alle stringhe *)
ord("a");
(* CORREZIONE *)
ord("#a");

(* ERRORE: explode prende in input una singola stringa, non una lista *)
explode ["bar"];
(* CORREZIONE *)
explode("bar");

(* ERRORE: implode accetta una lista di char, non una tupla *)
implode (#"a", #"b");
(* CORREZIONE (sostituisco le parentesi tonde con le quadre) *)
implode (["a", "b"]);

(* ERRORE: :: (cons) unisce UN elemento a una LISTA.
    Qui a sinistra c'è una lista e a destra un'altra lista *)
["r"]::["a","t"];
(* CORREZIONE: (rimuovo le parentesi a sinistra) *)
"r"::["a","t"];

```

4.2 Istanze di Tipi Complessi (Esercizio 1.6 - 1.7)

Si richiede di creare valori legali per le seguenti firme di tipo, dimostrando di saper annidare correttamente tuple (parentesi tonde) e liste (parentesi quadre).

```
(* TIPO: int list list list *)
(* Una lista che contiene liste di liste di interi *)
val es1 = [[[1, 2]], [[3]]];

(* TIPO: (int * char) list *)
(* Una lista di tuple formate da intero e carattere *)
val es2 = [(1, #"a"), (2, #"b")];

(* TIPO: string list * (int * (real * string)) *)
(* Una macro-tupla contenente una lista di stringhe e un'altra tupla *)
val es3 = (["ciao", "mondo"], (42, (3.14, "pi")));

(* TIPO: ((int * int) * (bool list) * real) * (real * string) *)
val es4 = (((1, 2), [true, false], 5.0), (3.14, "test"));
```

5 Regole di scope

5.1 Funzioni e inferenza di tipi

In ML, le funzioni sono considerate valori (cittadini di prima classe) a tutti gli effetti. Sono rappresentate tramite espressioni parametrizzate.

- **Sintassi base (fn):** La definizione di una funzione anonima si basa sulla sintassi `fn <parametro> => <espressione>`, che corrisponde concettualmente all'astrazione λ nel lambda calcolo. Esempio: `fn n => n + 1`.
- **Dichiarazioni con nome:** È possibile associare una funzione a un identificatore usando la parola chiave `val` (es. `val increment = fn n => n + 1`), ma ML offre anche un costrutto semplificato (zucchero sintattico) estremamente comune: `fun increment n = n + 1`.
- **Singolo parametro e Tuple:** In ML, **tutte le funzioni hanno esattamente un solo parametro**. Tuttavia, se abbiamo bisogno di passare molteplici argomenti logici, il parametro può essere strutturato come una tupla. Ad esempio, `fun max3(a:real, b, c) = ...` accetta in input un'unica tupla contenente tre elementi di tipo `real`.

Type Inference (Inferenza dei tipi): Il compilatore di ML deduce automaticamente i tipi delle variabili basandosi sul contesto delle espressioni:

- In un'espressione aritmetica come $(a + b) * 2.0$, sapendo che 2.0 è reale e l'operatore $*$ impone operandi compatibili, il risultato di $a+b$ e, di conseguenza, le variabili a e b vengono forzate e dedotte come tipo **real**.
- In presenza di costrutti condizionali, le espressioni associate ai rami **then** ed **else** e all'intera espressione condizionale stessa devono rigorosamente possedere lo stesso tipo.
- Qualora non vi siano operatori che formino un tipo specifico e non vi sia alcun modo per determinarne il tipo, il compilatore assegna un **tipo generico** (indicato per esempio con `'a` e `'b`).

5.2 Ambiente (Environment) e Blocchi

L'**Environment** (ambiente) è la collezione delle associazioni tra identificatori (nomi) e oggetti denotabili che si viene a creare a runtime in uno specifico momento del programma. Una *dichiarazione* è il meccanismo che crea tale associazione all'interno dell'ambiente. L'ambiente può essere logicamente suddiviso in:

- **Ambiente locale:** Variabili dichiarate internamente e parametri formali creati all'ingresso di un blocco.
- **Ambiente non-locale:** Associazioni ereditate da blocchi gerarchicamente esterni a quello corrente.
- **Ambiente globale:** Costituisce quella porzione dell'ambiente non-locale che contiene associazioni e dichiarazioni visibili in tutti i blocchi.

I **blocchi** (in ML definiti dal costrutto `let ... in ... end`) gestiscono l'organizzazione dell'ambiente. Una dichiarazione locale in un blocco è visibile nel blocco stesso e in tutti i blocchi annidati all'interno, a patto che in questi ultimi non vi sia un'ulteriore dichiarazione con il medesimo nome che ne occulta (shadowing) la precedente.

5.3 Scoping Statico vs Dinamico

Come viene risolto il riferimento a una variabile "non locale"? La letteratura dei linguaggi di programmazione definisce due approcci fondamentali:

- **Scoping Statico (o Lexical):** Il riferimento viene risolto nel blocco che **include testualmente (sintatticamente)** la chiamata. Si basa esclusivamente sulla struttura del codice scritto ed è indipendente dai nomi locali del blocco chiamante. I vantaggi dello scoping statico includono una maggiore facilità di lettura da parte del programmatore, migliore efficienza a runtime, e la possibilità per il compilatore di effettuare stringenti controlli di correttezza e ottimizzazioni. È adottato in linguaggi come C, Java, Pascal e dallo stesso ML.

- **Scoping Dinamico:** Il riferimento a un nome non-locale viene risolto all'interno del blocco che è **stato attivato per ultimo** durante il flusso di esecuzione (a runtime) e non ancora terminato. Tale approccio consente di alterare il comportamento di una funzione semplicemente modificando l'ambiente del chiamante, ma genera architetture fragili in cui i programmi diventano ostici da comprendere e meno efficienti.

6 Laboratorio ML 2

Nel secondo laboratorio si prendono i primi passi pratici scrivendo funzioni pure per elaborare primitive di ML e strutture complesse quali Tuple e Liste, verificandone il corretto type checking.

6.1 Esercizi base sulle Tuple

Esercizio L2.2 - Minimo di una tupla: Scrivere una funzione `min3` che restituisca l'elemento più piccolo all'interno di una tupla composta da tre numeri interi.

```
fun min3 (a: int, b: int, c: int) =
  if a < b andalso a < c then a
  else if b < c then b
  else c;
```

Esercizio L2.4 - Inversione di Tupla: Scrivere una funzione `reverse` che prenda in ingresso una tupla di 3 elementi invertendone gli oggetti (la funzione deve poter accettare Tuple eterogenee, es. `reverse(1.0, 2, "a")`).

```
fun reverse (a, b, c) = (c, b, a);
```

Esercizi L2.7 e L2.8 - Identificazione ed Ordinamento: Scrivere una funzione `min_max_pair` che data una tupla di 3 interi restituisca in uscita una nuova tupla (coppia) limitata al valore più piccolo e al valore più grande estratti. Successivamente, implementare una funzione `sort` in grado di restituire una lista ordinata degli elementi.

```
fun min_max_pair (a:int, b, c) =
  let
    val minimo = if a<b andalso a<c then a else if b<c then b else c
    val massimo = if a>b andalso a>c then a else if b>c then b else c
  in
    (minimo, massimo)
  end;
```

```
fun sort (a:int, b, c) =
  let
    val (minimo, massimo) = min_max_pair(a, b, c)
    val mediano = if (a <> minimo andalso a <> massimo) then a
                  else if (b <> minimo andalso b <> massimo) then b
                  else c
  in
    [minimo, mediano, massimo]
  end;
```

6.2 Esercizi base sulle Liste e sulle Stringhe

Esercizio L2.3 - Terzo elemento: Estrarre il terzo elemento da una lista di dati. L'esercizio prevede l'uso in sequenza delle primitive `hd` (head) e `tl` (tail) senza la necessità, in questa fase, di far operare correttamente la funzione su liste inferiori alla lunghezza prevista.

```
fun third L = hd(tl(tl(L)));
```

Esercizio L2.6 - Cycle: Scrivere una funzione `cycle` che prende una lista originaria `[a1, ..., an]` ed esegue un "giro", producendo come risultato la dislocazione `[a2, ..., an, a1]` (l'esercizio assume liste non vuote).

```
fun cycle L = tl(L) @ [hd(L)];
```

Esercizio L2.10 - Rimuovere il secondo elemento: Scrivere la funzione `rem` in grado di accettare in ingresso una lista e ritornare la struttura inalterata privata del secondo elemento (ad esempio, la lista `[1,2,3,4]` si traduce in `[1,3,4]`).

```
fun rem L = hd(L) :: tl(tl(L));
```

7 Astrazione del controllo e ricorsione

7.1 Spiegazione teorica

L'astrazione del controllo riguarda i meccanismi che i linguaggi di programmazione offrono per gestire il flusso di esecuzione (come blocchi condizionali, cicli e chiamate a sottoprogrammi). In ML, essendo un linguaggio funzionale, il controllo del flusso si basa pesantemente sulla **valutazione delle espressioni** e sulla **ricorsione**, a differenza dei linguaggi imperativi che si basano su costrutti iterativi (come il `while` o il `for`).

Metodi di passaggio dei parametri: Quando una funzione viene invocata, l'ambiente locale deve essere popolato associando i parametri formali ai parametri attuali. Esistono storicamente diverse strategie:

- **Call by value (Passaggio per valore):** I parametri attuali vengono valutati *prima* che la funzione venga chiamata. Il risultato viene copiato nel parametro formale. È un meccanismo sicuro e senza side-effect diretti sulle variabili esterne, ed è la strategia di default adottata da ML (eager evaluation).
- **Call by reference (Passaggio per riferimento):** Viene passato l'indirizzo di memoria del parametro attuale. Modificare il parametro formale nella funzione modifica la variabile originale (side-effect). Non è il meccanismo predefinito nei linguaggi funzionali puri.
- **Call by name / Call by need:** Il parametro non viene valutato finché non è strettamente necessario all'interno del corpo della funzione (lazy evaluation). Se non viene mai usato, non viene mai valutato, risparmiando computazione e prevenendo errori (come la divisione per zero).

Ricorsione: In assenza di cicli iterativi standard, la ricorsione è l'unico modo per compiere iterazioni in un linguaggio puramente funzionale. Il programmatore definisce un caso base (condizione di terminazione) e un passo ricorsivo (in cui la funzione invoca se stessa su un input ridotto). Un concetto fondamentale legato all'ottimizzazione è la **Tail Recursion** (Ricorsione in coda): si ha quando la chiamata ricorsiva è l'ultimissima operazione eseguita nel corpo della funzione. Questo permette al compilatore di riutilizzare lo stesso record di attivazione (Stack Frame) senza saturare la memoria (evitando lo Stack Overflow).

Mutua Ricorsione: Si verifica quando due o più funzioni si chiamano a vicenda. Nei linguaggi con regole di scope rigide (dove un nome deve essere definito prima di essere usato), definire due funzioni A e B che si chiamano a vicenda solleverebbe un errore: se dichiaro prima A, essa non conoscerà B, e viceversa. ML risolve elegantemente questo problema introducendo la parola chiave **and**, che permette di dichiarare e valutare le due funzioni contemporaneamente nello stesso blocco.

7.2 Implementazione del codice

7.2.1 Mutua Ricorsione in ML

Vogliamo scrivere due funzioni: **take**, che estrae gli elementi in posizione dispari da una lista, e **skip**, che salta gli elementi in posizione pari (concettualmente le due operazioni si alternano).

```
(* L'uso di 'and' permette la mutua ricorsione.
   Il compilatore valuta le firme di entrambe contemporaneamente. *)

fun take (L) =
  if L = nil then nil
  else hd(L) :: skip(tl(L))
and skip (L) =
  if L = nil then nil
  else take(tl(L));

(* Esempio di esecuzione *)
> take ([1, 2, 3, 4, 5]);
val it = [1, 3, 5] : int list
```

8 Laboratorio ML 3

Nel terzo laboratorio si consolida l'utilizzo della ricorsione semplice per risolvere problemi matematici e per manipolare liste, costruendo algoritmi iterativi attraverso chiamate a funzione ricorsive in stile dichiarativo.

8.1 Esercizi base sulla Ricorsione Matematica

Esercizio 3.1 - Fattoriale: Scrivere una funzione `fact` che calcola il fattoriale di un numero n (con $n \geq 1$). L'esercizio non richiede, in questa fase primaria, di gestire in modo sicuro i numeri negativi.

```
fun fact(n) =
  if n = 1 then 1
  else n * fact(n - 1);

(* Utilizzo: *)
> fact 10;
val it = 3628800 : int
(* Nota: In ML un "fact 100" genera una Exception-Overflow
   in quanto supera il valore massimo immagazzinabile nel tipo int *)
```

Esercizio 3.11 - Potenza (Power): Scrivere una funzione `power` che prenda in ingresso una tupla formata da un numero intero x e un esponente i (assunto non negativo) e ne calcoli la potenza x^i . Notare l'utilizzo del pattern matching semplificato nella firma della funzione.

```
fun power (x, 0) = 1
  | power (x, i) = x * power (x, i - 1);

(* Utilizzo: *)
> power (4, 3);
val it = 64 : int
```

8.2 Esercizi base sulla Ricorsione e Liste

Esercizio 3.10 - Duplicazione elementi (duplicatep): Scrivere una funzione che, data una lista di elementi di tipo qualsiasi, restituisca una nuova lista in cui ogni elemento originale è ripetuto consecutivamente due volte (es. da `[1,2]` a `[1,1,2,2]`). Questo esercizio introduce un'applicazione molto comune del pattern matching su liste.

```
(* L'uso dei pattern semplifica la scrittura rispetto all'uso degli if-else
   e di hd() / tl() *)
fun duplicatep (nil) = nil
  | duplicatep (x::xs) = x :: x :: duplicatep(xs);

(* Utilizzo: *)
> duplicatep [1, 2, 3, 4];
val it = [1, 1, 2, 2, 3, 3, 4, 4] : int list
```

Esercizio 3.12 - Massimo di una Lista (maxList): Scrivere una funzione per trovare l'elemento reale più grande all'interno di una lista (assunta non vuota). Questa funzione utilizza l'istruzione `let ... in ... end` per salvare il risultato della chiamata ricorsiva, evitandone il ricalcolo multiplo e ottimizzando le prestazioni (concetto propedeutico per strutture più avanzate).

```
fun maxList ([x:real]) = x
  | maxList (x::xs) =
    let
      val m = maxList(xs)
    in
      if x > m then x else m
    end;
```

9 Pattern ed environment locale in ML

9.1 Spiegazione teorica

In ML, oltre alla dichiarazione di funzioni tramite costrutti condizionali classici (`if-then-else`), è possibile (e fortemente consigliato) utilizzare il **Pattern Matching**. Questo meccanismo ricorda i costrutti `switch-case` dei linguaggi procedurali, ma è molto più potente perché permette di "destrutturare" i dati e assegnare variabili al volo.

- **Sintassi base dei pattern:** La definizione di una funzione procede elencando una sequenza di casi. Il primo pattern che corrisponde (esegue il "match") con l'argomento fornito determina quale espressione verrà valutata.

```
fun f (pattern_1) = espressione_1
  | f (pattern_2) = espressione_2
  | f (pattern_n) = espressione_n;
```

- **Esaustività:** La lista delle alternative dovrebbe sempre coprire tutti i casi possibili. Se i pattern non sono esaustivi, il compilatore di ML genera un *warning*. Se a runtime viene passato un argomento che non fa match con alcun pattern dichiarato, viene sollevata un'eccezione critica (`Exception- Match raised`).
- **Pattern validi e non validi:**
 - *Consentiti:* Costanti (es. `nil`, `0`, stringhe), costruttori di liste come l'operatore `cons` (es. `x::xs`, `x::y::xs`), tuple (es. `(x, y, z)`) e il carattere jolly o *wildcard* `_` (usato per i valori "anonimi" che non ci interessa memorizzare né usare).
 - *Non consentiti:* Operatori aritmetici (non si può scrivere `fun square(x+1) = ...`), concatenazione di liste tramite l'operatore `@`, numeri reali (`real`) e l'uso multiplo della stessa variabile in un singolo pattern (es. `fun uguali(n, n) = true` è illegale, la variabile `n` è già associata). L'identità logica va testata nel corpo con `if`.

Il costrutto `as`: Spesso è utile destrutturare un parametro (ad esempio separando testa e coda di una lista) ma al tempo stesso mantenere un riferimento "globale" all'intero oggetto originale per non doverlo ricostruire. La keyword `as` risolve questa necessità sintattica: scrivendo `L as x::xs`, la variabile `L` rappresenta la lista intera, mentre `x` e `xs` ne catturano rispettivamente testa e coda.

Costrutti `case` e `fn`: La sintassi del pattern matching si applica in modo identico anche alle funzioni anonime (`fn`) e al costrutto interno `case`:

```
case x of
  pattern_1 => espressione_1
| pattern_2 => espressione_2;
```

La differenza architetturale è che il `case` permette di effettuare pattern matching sul risultato momentaneo di un'espressione qualsiasi nel bel mezzo di una funzione, e non soltanto sui parametri di ingresso della stessa.

Ambiente locale (`let ... in ... end`): Per definire variabili temporanee, utilissime per memorizzare sotto-espressioni ricorrenti o decomporre i complessi risultati in uscita da una funzione, ML offre il costrutto `let`.

```
let
  val x = calcolo_pesante()
  val (a, b) = scorpora_tupla()
in
  a + b + x
end
```

I "binding" (assegnazioni) creati tra `let` e `in` arricchiscono l'ambiente locale ma restano visibili esclusivamente all'interno del blocco di codice delimitato tra `in` ed `end`.

9.2 Implementazione del codice

9.2.1 Merge e uso di `as`

Un esempio didattico eccellente in cui `as` semplifica la stesura è l'unione ordinata di due liste.

```
fun merge (nil, M) = M
  | merge (L, nil) = L
  | merge (L as x::xs, M as y::ys) =
    if x < y then x :: merge(xs, M)
    else y :: merge(L, ys);
```

Senza la keyword `as`, nel ramo `else` avremmo dovuto ricostruire a mano e sprecare risorse per ricreare la prima lista scrivendo `merge(x::xs, ys)`.

9.2.2 Utilizzo del `case` e *wildcard*

```
val is_one = fn x =>
  case x of
    1 => "one"
  | _ => "anything else";
```

10 Laboratorio ML 4

Il quarto laboratorio e i relativi esercizi di tutorato impongono l'abbandono delle funzioni primitive `hd` e `tl` viste all'inizio del corso, in favore dell'uso sistematico e ricorsivo del pattern matching. Le tecniche apprese qui costituiscono la base per affrontare le prove d'esame.

10.1 Destrutturazione avanzata di liste e tuple

Esercizio L4.1 - Flip alternato: Scrivere una funzione `flip` che inverte a due a due gli elementi di una lista e, se dispari, lascia l'ultimo elemento inalterato alla fine.

```
fun flip ([]) = []  
  | flip ([x]) = [x]  
  | flip (x::y::zs) = y :: x :: flip(zs);
```

Esercizio L4.4 - Flip di tuple in lista: Applicazione combinata di pattern matching su liste e tuple che sfrutta il costrutto `as` per risparmiare allocazioni di memoria.

```
fun flip (nil) = nil  
  | flip ((x as (a:int, b)) :: xs) =  
    if a < b then x :: flip(xs)  
    else (b, a) :: flip(xs);
```

Nota per l'esame: Se la tupla (a,b) è già posizionata nell'ordine corretto ($a < b$), inseriamo nella chiamata ricorsiva direttamente `x` (l'intera tupla catturata inalterata con `as`) senza sprecare risorse della macchina per ricreare inutilmente una nuova tupla (a,b) .

Esercizio dal Tutorato 3 - Separazione e Let-Val: Un grande classico d'esame: dividere una lista in due liste separate (una per i pari, una per i dispari), mantenendo l'ordine, esplorandola una sola volta.

```
fun separaPariDispari (nil) = (nil, nil)  
  | separaPariDispari (x::xs) =  
    let  
      (* Decompone elegantemente la tupla restituita dalla ricorsione *)  
      val (pari, dispari) = separaPariDispari (xs)  
    in  
      if x mod 2 = 0 then (x::pari, dispari)  
      else (pari, x::dispari)  
    end;
```

In questo frammento logico, il `let` evita la disastrosa pratica di calcolare due volte la ricorsione `separaPariDispari(xs)` (una volta per la testa e una volta per la coda).

10.2 Insiemi Logici tramite Liste (Esercizi L4.6 - L4.8)

Nel corso non esistono strutture dati pre-fabbricate per gli "Insiemi" (Set). Devono essere simulati e controllati manualmente garantendo l'assenza di duplicati logici.

Appartenenza (member) ed Eliminazione (delete): L'uso della *wildcard* `_` per il parametro non utilizzato (`"x"`) nel caso base previene i *warning* di tipo.

```
fun member (_, nil) = false
  | member (x, y::ys) = (x = y orelse member(x, ys));

fun delete (a, []) = []
  | delete (b, c::ys) =
    if b = c then ys
    else c :: delete(b, ys);
```

Inserimento con controllo di univocità (insert): Anche qui l'efficienza si ottiene combinando l'ambiente locale tramite l'identificatore strutturato.

```
fun insert (x, nil) = [x]
  | insert (x, S as y::ys) =
    if x = y then S
    else y :: insert(x, ys);
```

11 Input-output, eccezioni e funzioni polimorfe

11.1 Spiegazione teorica

Questa sezione affronta la gestione delle operazioni di input/output, il sistema di controllo e recupero degli errori a runtime tramite le eccezioni, e i fondamenti del polimorfismo nel sistema di tipi di ML.

Input e Output in ML: A differenza della valutazione pura di espressioni matematiche, le operazioni di I/O introducono nel linguaggio il concetto di **effetto collaterale** (side-effect), modificando lo stato dei canali di comunicazione standard o dei file memorizzati su disco.

- **Output basico:** La funzione predefinita `print` accetta come argomento una stringa e la stampa sullo standard output. La funzione possiede la firma `string -> unit`; restituisce l'unico valore del tipo `unit`, ovvero la tupla vuota `()`, indicando che l'espressione ha esaurito la sua computazione producendo unicamente un effetto collaterale a video.
- **Espressioni composte:** Per eseguire una sequenza ordinata di operazioni che causano effetti collaterali, ML permette l'uso di espressioni composte separate dal punto e virgola e racchiuse tra parentesi tonde: `(exp1; exp2; ...; expn)`. Il tipo e il valore finale dell'intera macro-espressione sono determinati unicamente dall'ultima sotto-espressione valutata (`expn`). Le istruzioni intermedie servono esclusivamente per i loro effetti collaterali e l'ultima istruzione non necessita del punto e virgola di chiusura.

- **Gestione dei File (TextIO):** ML offre supporto linguistico all'interazione con il file system tramite la struttura standard `TextIO`. I canali di lettura vengono gestiti tramite oggetti di tipo `TextIO.instream`. Le funzioni principali includono:
 - `TextIO.openIn(filename)`: Apre un file di testo in modalità lettura e restituisce un canale `instream`.
 - `TextIO.endOfStream(infile)`: Restituisce un valore booleano che indica se si è raggiunta la fine del file.
 - `TextIO.inputN(infile, n)`: Legge una quantità fissa pari a `n` caratteri dal file, restituendoli sotto forma di stringa.
 - `TextIO.input(infile)`: Legge l'intero contenuto rimanente del file come un'unica stringa.
 - `TextIO.inputLine(infile)`: Legge una riga di testo comprensiva del carattere di newline `\n`. Restituisce un tipo speciale costruttore denominato **option**.

Il tipo costruttore option: In presenza di funzioni che possono non restituire un valore valido (come la lettura oltre la fine di un file o la ricerca in un dizionario), ML evita l'uso di puntatori nulli fragili introducendo il tipo parametrico `'a option`. Un oggetto di tipo `option` può assumere due forme:

- **SOME v:** Contiene il valore effettivo `v` di tipo `'a`.
- **NONE:** Indica l'assenza di un valore valido (es. fine del file raggiunta).

Per estrarre il valore da una variabile `option` forzandone il tipo da `'a option` ad `'a`, si utilizza la funzione di sistema `valOf`. Se applicata su un elemento che contiene **SOME v**, restituisce `v`; se applicata su **NONE**, interrompe il programma sollevando l'eccezione **Option**.

Eccezioni in ML: Le eccezioni sono costrutti linguistici semantici per interrompere il flusso normale del programma in presenza di errori a runtime.

- **Eccezioni di sistema:** Sollevate automaticamente dalla macchina astratta di ML. Esempi comuni sono **Div** (divisione per zero), **Empty** (applicazione di `hd` o `tl` su liste vuote) e **Chr** (conversione ASCII fuori range).
- **Eccezioni utente e parole chiave:** Il programmatore può definire nuove eccezioni nell'ambiente tramite la keyword `exception Nome`. Le eccezioni possiedono un tipo primitivo dedicato chiamato `exn`. Per innescare materialmente un errore si usa la keyword `raise Nome`.
- **Eccezioni con parametri:** È possibile associare un contenuto informativo all'eccezione definendola tramite la sintassi `exception Nome of tipo`. In questo caso l'identificatore agisce da costruttore (es. `exception Errore di int`, lanciata con `raise Errore(42)`).
- **Gestione delle eccezioni (handle):** Per intercettare un errore e ripristinare il flusso si appone l'operatore `handle` a valle di un'espressione, implementando una sintassi analoga al pattern matching: `<espressione> handle PatternEccezione => espressione_di_recupero`.

Funzioni polimorfe in ML: Il termine polimorfismo indica la capacità di un frammento di codice di operare su molteplici forme e tipi di dato. ML adotta un approccio statico rigoroso (*compile-time strong typing*), ma implementa il polimorfismo parametrico: se una funzione non esegue operazioni esclusive di un tipo specifico, il compilatore le assegna una variabile di tipo generica indicata con un singolo apice iniziale ('a, 'b).

Interazione tra Operatori e Polimorfismo:

- **Operatori che restringono il polimorfismo:** Gli operatori aritmetici classici (+, -, *), gli operatori di divisione (/, div, mod), i confronti di disuguaglianza (<, >), i connettivi booleani e le funzioni di conversione esplicita forzano i parametri ad assumere tipi primitivi rigidi (es. `int` o `real`), inibendo il polimorfismo.
- **Operatori che preservano il polimorfismo:** Gli operatori sulle tuple (creazione e proiezioni `#n`), gli operatori nativi sulle liste (`::`, `@`, `hd`, `tl`, `nil`) e gli operatori di uguaglianza logica consentono l'uso di tipi generici parametrici.
- **Tipi di uguaglianza (Equality types):** Gli operatori = e <> richiedono che i tipi messi a confronto supportino la verifica di uguaglianza. Sono considerati tipi di uguaglianza gli interi, i booleani, i caratteri, le stringhe e le tuple/liste composte dagli stessi. Sono strutturalmente **esclusi** i numeri reali (`real`) e tutte le funzioni. Le variabili di tipo generico vincolate a supportare esclusivamente l'uguaglianza vengono identificate dal compilatore con un **doppio apice iniziale** ("a). Cercare di applicare funzioni con vincolo "a su numeri reali genera un errore statico di unificazione del tipo.
- **Osservazione critica sull'inversione di liste:** Se si scrive una funzione di inversione testando la fine della lista tramite il controllo booleano `if L = nil then ...`, l'uso dell'operatore = costringe il compilatore ad assegnare alla funzione il tipo vincolato "a list -> "a list. Di conseguenza, la funzione fallirà il controllo dei tipi se applicata a liste di numeri reali o di funzioni. Per preservare il polimorfismo puro ('a list -> 'a list), si deve ricorrere al pattern matching sui costruttori della lista o utilizzare la funzione predefinita `null(L)`.

11.2 Implementazione del codice

11.2.1 Esempio di I/O e gestione dei flussi di lettura da File

(* Funzione ricorsiva ausiliaria che consuma lo stream carattere per carattere *)

```
fun makeList1 (infile, NONE) = nil
  | makeList1 (infile, SOME c) =
    c :: makeList1(infile, TextIO.input1(infile));
```

(* Inizializza la lettura estraendo il primo carattere *)

```
fun makeList (infile) = makeList1(infile, TextIO.input1(infile));
```

(* Utilizzo completo con apertura e chiusura del canale *)

```
fun readFileAndConvert (filename) =
  let
    val instream = TextIO.openIn(filename)
    val charList = makeList(instream)
  in
    TextIO.closeIn(instream); (* Effetto collaterale di chiusura *)
    charList                  (* Restituisce la lista di caratteri *)
  end;
```

11.2.2 Eccezioni personalizzate e meccanismi di recupero (handle)

```
exception OutOfRange of int * int;
```

```
fun comb1 (n, m) =
  if n <= 0 then raise OutOfRange(n, m)
  else if m < 0 orelse m > n then raise OutOfRange(n, m)
  else if m = 0 orelse m = n then 1
  else comb1(n - 1, m) + comb1(n - 1, m - 1);
```

(* La funzione comb fa da wrapper gestendo l'eccezione tramite handle *)

```
fun comb (n, m) =
  comb1(n, m)
  handle OutOfRange(0, 0) => 1
    | OutOfRange(n, m) =>
      (print("Error: values out of range. n=" ^ Int.toString(n) ^
        " m=" ^ Int.toString(m) ^ "\n");
       0);
```

11.2.3 Polimorfismo puro vs Polimorfismo di uguaglianza

```
(* Funzione con polimorfismo puro: accetta qualsiasi tipo *)
fun identity (x) = x;
(* Tipo assegnato: val identity = fn: 'a -> 'a *)

(* Funzione con polimorfismo vincolato all'uguaglianza *)
fun identity_eq (x) = if (x = x) then x else x;
(* Tipo assegnato: val identity_eq = fn: ''a -> ''a *)

(* Funzione di inversione che preserva il polimorfismo totale *)
fun rev_polymorphic (nil) = nil
  | rev_polymorphic (x::xs) = rev_polymorphic(xs) @ [x];
(* Tipo assegnato: val rev_polymorphic = fn: 'a list -> 'a list *)
```

12 Laboratorio ML 5

Il quinto laboratorio si concentra sull'ottimizzazione delle funzioni ricorsive tramite l'uso efficiente dell'ambiente locale (`let val`), sull'analisi delle espressioni di controllo condizionale annidate (`case`) e sull'implementazione pratica di funzioni che generano effetti collaterali tramite la stampa a video.

12.1 Ottimizzazione dell'Ambiente Locale (`let val`)

Esercizio L5.4 - Insieme delle parti (`powerSet`) efficiente: Rispetto alla versione base in cui la chiamata ricorsiva sulla coda della lista veniva calcolata due volte raddoppiando la complessità a ogni passo, l'introduzione di un legame temporaneo nel `let` riduce drasticamente i tempi di calcolo memorizzando il risultato parziale.

```
(* Funzione di supporto per inserire un elemento in tutte le sotto-liste *)
fun insertAll (x, nil) = nil
  | insertAll (x, y::ys) = (x::y) :: insertAll(x, ys);

fun powerSet (nil) = [nil]
  | powerSet (x::xs) =
    let
      val tailPowerSet = powerSet(xs) (* Calcolata una sola volta *)
    in
      tailPowerSet @ insertAll(x, tailPowerSet)
    end;
```

Esercizio L5.7 - Computazione efficiente di x^{2^i} (doubleExp): Scrivere un programma che calcoli la potenza doppia effettuando una sola chiamata ricorsiva per ogni decremento dell'indice i .

```
fun doubleExp (x: real, 0) = x
  | doubleExp (x, i) =
    let
      val y = doubleExp(x, i - 1)
    in
      y * y
    end;
```

Esercizio L5.3 - Destrutturazione di Tuple senza Pattern Matching diretto: Scrivere la funzione `split` estraendo i valori della tupla risultante tramite l'uso esplicito delle funzioni di proiezione `#1` e `#2` all'interno del blocco locale.

```
fun split (nil) = (nil, nil)
  | split ([a]) = ([a], nil)
  | split (a::b::cs) =
    let
      val tuplaRisultato = split(cs)
      val M = #1 tuplaRisultato
      val N = #2 tuplaRisultato
    in
      (a::M, b::N)
    end;
```

12.2 Uso delle espressioni case

Esercizio L5.1 - Simulazione dell'if-then-else tramite case: Implementare la funzione `is_lower_than5` verificando la validità della condizione logica booleana tramite pattern matching interno.

```
fun is_lower_than5 n =
  case (n < 5) of
    true => 1
  | false => 2;
```

12.3 Esercizi di Input/Output e Stampa

Esercizio L5.9 - Stampa di una lista (printList): Scrivere una funzione che accetti una lista di interi e la stampi riga per riga sullo standard output.

```
fun printList (nil) = ()
  | printList (x::xs) =
    (
      print(Int.toString(x));
      print("\n");
      printList(xs)
    );
```

Esercizio L5.10 - Calcolo e Formattazione Combinazioni (comb): Scrivere una funzione che calcoli il coefficiente binomiale stampando a video in modo strutturato i parametri inseriti e il risultato finale ottenuto.

```
fun factorial 0 = 1
  | factorial n = n * factorial(n - 1);

fun comb (n, m) =
  (
    print("n is ");
    print(Int.toString(n));
    print("\n");
    print("m is ");
    print(Int.toString(m));
    print("\n");
    print("Result is ");
    print(Int.toString(factorial(n) div (factorial(m) * factorial(n - m))));
    print("\n")
  );
```

13 Funzioni di ordine superiore e curried

13.1 Spiegazione teorica

In ML le funzioni sono trattate come "cittadini di prima classe", il che significa che possono essere assegnate a variabili, passate come argomenti ad altre funzioni o restituite come risultato. Una funzione che accetta o restituisce un'altra funzione prende il nome di **Funzione di Ordine Superiore** (Higher-Order Function).

Gestione dell'Ambiente e Binding: Quando si passa una funzione come parametro o la si restituisce in uscita, sorge un problema architetturale: quale ambiente non-locale deve essere utilizzato durante la sua futura esecuzione?

- **Deep binding:** L'ambiente non-locale viene "congelato" al momento della *creazione* (o passaggio) della funzione. È l'approccio adottato in accoppiata allo *Scoping Statico*.
- **Shallow binding:** L'ambiente non-locale viene valutato solo al momento in cui la funzione viene effettivamente *chiamata*.

Come già visto, ML utilizza rigorosamente lo **Scoping Statico** abbinato al **Deep binding**. Questa combinazione viene tecnicamente implementata attraverso le *Closure*, ovvero strutture dati nascoste che memorizzano il codice della funzione e l'ambiente al momento della sua definizione, garantendone la robustezza e la prevedibilità.

Funzioni Curried (Currying): A livello formale e sintattico, **ogni funzione in ML accetta uno e un solo argomento**. Fino a questo momento abbiamo aggirato questo limite matematico passando argomenti multipli impacchettati all'interno di una tupla (es. `fun max(x, y)`). Un approccio alternativo, proprio dei linguaggi puramente funzionali, è la **forma Curried** (in onore del logico Haskell Curry). Una funzione a due argomenti viene progettata in modo tale da prendere il primo argomento e *restituire una nuova funzione* che attenderà il secondo.

- *Sintassi Tupla:* `fun exp1(x, y) = ...` ha tipo `real * int -> real`.
- *Sintassi Curried:* `fun exp2 x y = ...` ha tipo `real -> int -> real` (valutato a destra come `real -> (int -> real)`).
- **Vantaggio: Applicazione Parziale (Partial Instantiation).** Le funzioni curried permettono di fornire soltanto una parte degli argomenti richiesti, generando istantaneamente una nuova funzione "specializzata" da riutilizzare a runtime. Ad esempio, potremmo fissare la base di una potenza a 3.0 e usare la nuova funzione creata solo per far variare l'esponente.

13.2 Implementazione del codice

13.2.1 Funzioni di Ordine Superiore Classiche (Implementate a mano)

Costruiamo le versioni semplificate delle classiche funzioni funzionali per comprendere il loro comportamento interno (il linguaggio ML dispone già di versioni built-in altamente ottimizzate).

```
(* 1. simpleMap: applica la funzione F a tutti gli elementi di una lista *)
fun simpleMap (F, nil) = nil
  | simpleMap (F, x::xs) = F(x) :: simpleMap(F, xs);
(* Tipo: ('a -> 'b) * 'a list -> 'b list *)

(* 2. filter: mantiene solo gli elementi che soddisfano il predicato P *)
fun filter (P, nil) = nil
  | filter (P, x::xs) =
    if P(x) then x :: filter(P, xs)
    else filter(P, xs);
(* Tipo: ('a -> bool) * 'a list -> 'a list *)

(* 3. reduce: collassa la lista applicando ricorsivamente la funzione F *)
exception EmptyList;
fun reduce (F, nil) = raise EmptyList
  | reduce (F, [a]) = a
  | reduce (F, x::xs) = F(x, reduce(F, xs));
(* Tipo: ('a * 'a -> 'a) * 'a list -> 'a *)
```

13.2.2 Currying e Applicazione Parziale

```
(* Funzione in forma Curried, notare l'assenza di virgole e parentesi *)
fun exponent2 x 0 = 1.0
  | exponent2 x y = x * exponent2 x (y - 1);

(* Possiamo chiamarla fornendo subito entrambi i parametri... *)
> exponent2 3.0 4;
val it = 81.0 : real

(* ...oppure possiamo fornire solo il primo parametro (la base 3.0),
   ottenendo in ritorno una NUOVA funzione pronta a ricevere un esponente *)
> val powerOfThree = exponent2 3.0;
val powerOfThree = fn : int -> real

> powerOfThree 4;
val it = 81.0 : real
```

14 Laboratorio ML 6

Nel sesto laboratorio si fondono le nozioni di I/O (File e testuali) introdotte marginalmente nella Lezione 7, assieme alle Funzioni di Ordine Superiore, per costruire elaboratori dati puramente funzionali.

14.1 Elaborazione di Flussi di Testo (I/O)

Viene esplorata l'interazione con l'esterno usando la struttura `TextIO`, supportata dai tipi costruttori come `option` (per gestire letture di fine file che restituiranno `NONE`) e dal pattern matching per il disinnescamento degli errori.

Esercizio base: Primitive TextIO

```
(* 1. Aprire un file in lettura (restituisce un instream) *)
val input = TextIO.openIn("file.txt");

(* 2. Leggere i successivi 5 caratteri (restituisce string) *)
TextIO.inputN(input, 5);

(* 3. Leggere una riga intera (restituisce string option) *)
TextIO.inputLine(input);

(* 4. Leggere il prossimo carattere senza consumarlo dal flusso *)
TextIO.lookahead(input);

(* 5. Chiudere rigorosamente il flusso per liberare la risorsa *)
TextIO.closeIn(input);
```

14.2 Esercizi dal Tutorato: I/O Sicuro ed Eccezioni

Nel tutorato viene richiesto di costruire dei gestori di file incapsulati che intercettano gli errori di sistema per lanciare eccezioni di dominio semantico.

Esercizio: Leggere la prima riga in sicurezza Scrivere una funzione che tenti di leggere la prima riga di un file. Se il file non esiste, bisogna catturare l'eccezione di sistema di basso livello (`Io`) per sollevarne una propria (`FileNotFound`). Se il file esiste ma è vuoto, lanciare `EmptyFile`.

```
exception FileNotFound of string;
exception EmptyFile;
fun readFirstLine(filename) =
  let
    (* Tentativo di apertura controllato tramite handle *)
    val file = TextIO.openIn(filename)
        handle Io => raise FileNotFound(filename)

    val lineOpts = TextIO.inputLine(file)
  in
    TextIO.closeIn(file);
    (* Valutazione polimorfica dell'option per l'esito della lettura *)
    case lineOpts of
      NONE => raise EmptyFile
    | SOME(line) => line
  end;
```

14.3 Combinare I/O e Ordine Superiore

Esercizio L6.9 - Lettura e Somma (readAndSum): Scrivere una funzione che legga un file di testo contenente un numero (stringa) per ogni riga, ne esegua il parsing ad intero e ne calcoli la somma totale. Si richiede l'uso della funzione nativa `Int.fromString` che trasforma stringhe in interi, restituendo un `int option` di sicurezza.

```
fun readAndSum1 file =
  if TextIO.endOfStream(file) then 0
  else
    (* valOf forza l'estrazione "scartando" il costruttore SOME,
       estraendo il valore reale. Si usa due volte:
       per l'inputLine e per fromString *)
    valOf(Int.fromString(valOf(TextIO.inputLine(file))))
    + readAndSum1(file);

fun readAndSum filename =
  let
    val f = TextIO.openIn(filename)
    val res = readAndSum1(f)
  in
    TextIO.closeIn(f); (* Chiusura garantita *)
    res
  end;
```

Esercizio L6.10 - Lista di funzioni curried (applyList): Un eccellente esercizio in forma curried: scrivere una funzione che accetti prima una lista di funzioni (es. elevamento al quadrato, al cubo) e in seguito un valore `a`, restituendo una lista in cui ogni funzione è stata iterativamente applicata al valore di partenza.

```
(* Funzione Curried: prende prima la lista Fs, poi si "congela"
   aspettando che venga fornito il valore 'a' *)
fun applyList nil a = nil
  | applyList (F::Fs) a = F(a) :: (applyList Fs a);

(* Esempio di utilizzo con istanziazione finale: *)
> applyList [fn x => x * 2, fn x => x * x * x] 4;
val it = [8, 64] : int list
```

15 Tipi di dati e composizione di funzioni

15.1 Spiegazione teorica

In questa lezione si conclude la panoramica sulle funzioni di ordine superiore esaminando le primitive di libreria, l'operatore di composizione e, soprattutto, l'architettura per la definizione di nuovi tipi di dato e strutture ricorsive.

Funzioni di Ordine Superiore di Sistema (Built-in): Le funzioni di ordine superiore fornite nativamente da ML (come `map`, `foldr` e `foldl`) sono **curried**, ovvero si aspettano gli argomenti passati in sequenza (separati da spazio) e non raggruppati in una singola tupla.

- **map:** Applica una determinata funzione a ciascun elemento di una lista, restituendo una nuova lista con i risultati.
- **foldr (Fold Right):** "Collassa" una lista in un singolo valore combinando gli elementi da destra verso sinistra, partendo da un valore iniziale base. Corrisponde logicamente all'applicazione annidata: $f(x_1, f(x_2, \dots f(x_n, c)))$.
- **foldl (Fold Left):** Applica l'aggregazione partendo da sinistra verso destra. È spesso più efficiente poiché strutturabile tramite *tail recursion*. Corrisponde a: $f(x_n, f(x_{n-1}, \dots f(x_1, c)))$.

Composizione di Funzioni: La composizione matematica di due funzioni, $H(x) = G(F(x))$, viene supportata in ML dall'operatore infisso `o` (una semplice lettera 'o' minuscola). La sintassi `G o F` crea un'unica nuova funzione in cui l'output di `F` viene reindirizzato come input di `G`. È una tecnica dichiarativa fondamentale per concatenare manipolazioni di liste ed elaborazioni dati in catene di montaggio algoritmiche (*pipeline*).

Il Sistema dei Tipi in ML: Per estendere i tipi predefiniti (`int`, `real`, `string`, liste e tuple), ML mette a disposizione due costrutti principali:

- **type (Ridenominazione e Alias):** Permette di creare un alias per un tipo già esistente o per un'associazione complessa, con lo scopo di rendere il codice più leggibile per il programmatore. Non genera veri e propri tipi nuovi per la macchina. Sintassi base: `type signal = int list`; oppure `type ('a, 'b) mapping = ('a * 'b) list`;

- **datatype (Tipi e Costruttori Strutturati):** Introduce tipologie di dato interamente nuove. Costituisce la base per i *Tipi Unione* (*Union Types* o ADT). Un **datatype** definisce un **costruttore di tipo** (il nome generale) e uno o più **costruttori di dato** (i valori validi, separati dal pipe |). I costruttori di dato possono anche "trasportare" un parametro, permettendo di incapsulare valori (es. `S of 'a`).

Tipi di dato Ricorsivi: Poiché i costruttori possono riferirsi al **datatype** stesso che stanno definendo, questo strumento linguistico è ideale per implementare **Alberi Binari**, liste concatenate custom o altre strutture dati dinamiche non lineari. Per generare **tipi mutuamente ricorsivi** (come un Albero la cui definizione dipende da una Foresta e viceversa), si connettono le rispettive dichiarazioni utilizzando la keyword **and**.

15.2 Implementazione del codice

15.2.1 Composizione e Primitive List

```
(* Utilizzo di map per estrarre la lunghezza delle stringhe *)
> map size ["La", "lezione", "nove"];
val it = [2, 7, 4] : int list

(* Utilizzo combinato di foldr per riprodurre un Logical AND su lista *)
> val andb = foldr (fn (x,y) => x andalso y) true;
val andb = fn : bool list -> bool
> andb [true, false, true];
val it = false : bool

(* Composizione elegante: converte i char in stringhe e li concatena *)
> val implode = (foldr (op ^) "") o (map str);
val implode = fn : char list -> string
```

15.2.2 Costrutti Datatype e Union Types

```
(* Tipo Unione Base: Enumerazione *)
> datatype fruit = Apple | Pear | Grape;

(* Uso pratico dell'Unione col Pattern Matching *)
> fun isApple (x) = (x = Apple);
val isApple = fn : fruit -> bool

(* Tipo Elemento Generico: può contenere un Pair o un Valore Singolo *)
> datatype ('a, 'b) element = P of 'a * 'b | S of 'a;
> P("a", 1);
val it = P ("a", 1) : (string, int) element

(* Somma sicura ignorando gli elementi singoli S *)
fun sumElList (nil) = 0
  | sumElList (S(x)::L) = sumElList(L)
  | sumElList (P(x,y)::L) = y + sumElList(L);
```

16 Laboratorio ML 7

Nel settimo laboratorio si sperimenta materialmente l'utilizzo delle funzioni curried e la definizione algoritmica di esplorazione degli Alberi Binari creati attraverso i `datatype`.

16.1 Currying e Composizione Pratica (Esercizio L7.1 - L7.4)

Scrivere una funzione che prende una funzione non curried (una che lavora sulle tuple) e la trasforma nella sua versione curried, separandone gli argomenti in chiamate ad anello.

```
(* Definizione di un generalizzatore Curry (arit  3) *)
fun curry F x1 x2 x3 = F(x1, x2, x3);

(* Utilizzo del trasformatore *)
> curry (fn (x,y,z) => x * y * z);
val it = fn : int -> int -> int -> int -> int

(* Assegnazione tramite Applicazione Parziale *)
> val G = curry (fn (x,y,z) => x * y * z) 1 2;
> G 3;
val it = 6 : int
```

16.2 Tipi di dato per gli Alberi Binari (Esercizi L7.5 - L7.10)

L'albero binario generico per tutti gli esercizi di questa sezione viene cos  codificato:

```
datatype 'a btree = Empty
                  | Node of 'a * 'a btree * 'a btree;
```

Esercizi L7.5 e L7.6 - Visita PostOrder e InOrder: Implementare le canoniche funzioni di esplorazione degli alberi riordinando le chiamate ricorsive e ricomponendo i rami tramite concatenazione di liste (@).

```
fun postOrder (Empty) = nil
  | postOrder (Node(a, left, right)) =
    postOrder(left) @ postOrder(right) @ [a];

fun inOrder (Empty) = nil
  | inOrder (Node(a, left, right)) =
    inOrder(left) @ [a] @ inOrder(right);
```

Esercizio L7.8 e L7.10 - Calcolo e Manipolazione (sumTree e doubleTree):
 Sommare le componenti o alterare l'intero strato dei nodi clonando strutturalmente l'albero.
 Nel primo esempio si assume un albero che contiene tuple di tipo `string * int` come etichette dei nodi (*mapTree*).

```
(* Somma la seconda componente (b) delle coppie di label *)
fun sumTree Empty = 0
  | sumTree (Node((a, b), left, right)) =
    b + sumTree(left) + sumTree(right);

(* Raddoppia i valori producendo un albero binario della medesima forma *)
fun doubleTree Empty = Empty
  | doubleTree (Node(x, left, right)) =
    Node(x * 2, doubleTree(left), doubleTree(right));
```

16.3 Esercizi e approfondimenti dal tutorato: Analisi di Struttura sugli Alberi

Le simulazioni d'esame e le sessioni di tutorato (es. schede sugli Alberi Binari) arricchiscono le capacità di investigare la topologia dell'albero e valutarne gli attributi matematici (Alberi Bilanciati e Perfetti).

Esercizio: Altezza e Albero Perfetto (isPerfect) Un albero è perfetto se tutte le foglie giacciono sulla stessa identica profondità e ogni nodo non foglia ha precisamente due figli. Si deve valutare coerentemente l'uguaglianza tra i rami sottostanti.

```
(* Funzione di supporto vitale per l'analisi della topologia *)
fun height Empty = 0
  | height (Node(_, l, r)) = 1 + Int.max(height(l), height(r));

fun isPerfect Empty = true
  | isPerfect (Node(_, l, r)) =
    height(l) = height(r) andalso isPerfect(l) andalso isPerfect(r);
```

Esercizio: Albero Bilanciato (`isBalanced`) Un albero è bilanciato se la profondità dei due sotto-alberi relativi a un qualsiasi nodo differisce in altezza, al massimo, per un valore assoluto di 1 (analogia con gli *Alberi AVL*). La soluzione proposta previene uno scorrimento inefficiente adottando la restituzione di `~1` come codice di errore di sbilanciamento (Fail-Fast pattern interno).

```
fun isBalanced t =
  let
    (* Restituisce l'altezza, o ~1 nel caso un ramo fosse sbilanciato *)
    fun checkBal Empty = 0
      | checkBal (Node(_, l, r)) =
        let
          val hl = checkBal(l)
          val hr = checkBal(r)
        in
          if hl = ~1 orelse hr = ~1 orelse abs(hl - hr) > 1 then ~1
          else 1 + Int.max(hl, hr)
        end
      end
  in
    checkBal(t) <> ~1
  end;
```

Nota di ottimizzazione architetturale per l'esame: Calcolare sistematicamente `height(1)` e poi chiamare di nuovo `isBalanced(1)` provocherebbe un costo esponenziale sull'attraversamento (l'albero verrebbe scorso ripetutamente ai livelli bassi). Questa funzione annidata incapsula l'esplorazione restituendo entrambe le informazioni in una *singola* passata post-order, scalando la complessità ad $O(N)$.

Esercizio: Albero Specchio (`mirrorTree`) Data una radice asimmetrica, restituire il corrispettivo speculare ribaltando sistematicamente l'orientamento foglia-per-foglia.

```
fun mirrorTree Empty = Empty
  | mirrorTree (Node(v, l, r)) = Node(v, mirrorTree(r), mirrorTree(l));
```

17 Abstract Data Types in ML e Funtori

17.1 Spiegazione teorica

Per costruire sistemi software scalabili e sicuri, i linguaggi moderni offrono meccanismi di **Astrazione dei Dati** (Data Abstraction). L'obiettivo è separare nettamente l'*interfaccia* (cosa fa un tipo di dato e quali operazioni supporta) dalla sua *implementazione* (come i dati sono materialmente rappresentati in memoria).

In ML, questo incapsulamento (simile al concetto di classe e interfaccia in Java) viene realizzato attraverso tre costrutti fondamentali: **Signature**, **Structure** e **Functor**.

1. Signatures (Firme): Una *signature* agisce come un contratto pubblico o un'interfaccia. Definisce i tipi, le eccezioni e le firme delle funzioni che una struttura dovrà obbligatoriamente implementare.

- Sintassi: `signature NOME_SIG = sig ... end`
- Non contiene alcuna logica o codice eseguibile, ma solo dichiarazioni (`val`, `type`, `exception`).

2. Structures (Strutture): Una *structure* è l'implementazione concreta (il modulo) che fornisce il codice per i tipi e i metodi. Si collega a una signature per garantirne la validità strutturale (Ascription).

- **Ascription Trasparente (:):** `structure S : SIG = struct ... end`. Il compilatore controlla che *S* rispetti *SIG*. Tuttavia, l'implementazione interna dei tipi rimane visibile all'esterno (violando parzialmente l'incapsulamento).
- **Ascription Opaca (:>):** `structure S :> SIG = struct ... end`. È l'astrazione dei dati per eccellenza. Tutto ciò che non è esplicitamente definito nella signature viene nascosto. Se il tipo interno è implementato come `int`, all'esterno verrà visto solo come tipo astratto inaccessibile, impedendo all'utente di farvi operazioni aritmetiche illecite.

3. Functors (Funtori): Un funtore è una "struttura parametrizzata", ovvero una funzione che opera a livello di moduli. Prende in input una o più *structure* (che rispettano determinate *signature*) e genera in output una nuova *structure*.

- Sono il meccanismo principale in ML per la riusabilità del codice. Ad esempio, si può scrivere un funtore che genera un Albero Binario di Ricerca (BST) generico, passando in input una struttura che definisce semplicemente la funzione logica di ordinamento (es. alfabetico o numerico).

17.2 Implementazione del codice: Trasparenza vs Opacità

Il seguente frammento (analizzato nel Lab 9) dimostra la potenza dell'ascription opaca per proteggere l'invariante di un tipo astratto.

```
(* 1. La Firma nasconde l'implementazione del tipo 't' *)
signature MY_INT = sig
  type t
  val crea: int -> t
end;

(* 2. Struttura Trasparente: espone i dettagli *)
structure Transparent : MY_INT = struct
  type t = int
  fun crea x = x
end;

(* 3. Struttura Opaca (ADT Puro): blinda i dettagli *)
structure Opaque :> MY_INT = struct
  type t = int
  fun crea x = x
end;

(* Test di violazione dell'astrazione: *)
> val a = Transparent.crea(5);
> val b = a + 1; (* OK! Il compilatore sa che 'a' è un int *)

> val c = Opaque.crea(5);
> val d = c + 1;
(* ERRORE FATALE! Il compilatore blocca l'operazione.
   Per l'esterno, 'c' è di tipo astratto 't', non un 'int',
   quindi l'operatore '+' non è applicabile. *)
```

18 Laboratorio ML 8 - Alberi Binari di Ricerca

Il Laboratorio 8 e le relative schede di tutorato applicano le strutture astratte all'implementazione del **Binary Search Tree (BST)**, una struttura dati cruciale per la ricerca efficiente. La regola invariante di un BST stabilisce che per ogni nodo, tutti i valori nel sotto-albero sinistro devono essere strettamente minori del nodo, e tutti quelli nel sotto-albero destro strettamente maggiori.

18.1 Creazione di un BST tramite Functor (MakeBST)

Per rendere l'albero riutilizzabile per interi, stringhe o record personalizzati, lo si genera tramite un funtore che riceve in input l'operatore di confronto `lt` (less than).

```
(* Step 1: Signature che definisce il contratto di ordinamento *)
signature TOTALORDER = sig
    type t
    val lt : t * t -> bool
end;

(* Step 2: Struttura concreta per stringhe *)
structure StringOrder : TOTALORDER = struct
    type t = string
    fun lt (a, b) = a < b
end;

(* Step 3: Funtore per costruire l'albero *)
functor MakeBST (Order : TOTALORDER) = struct
    datatype bst = Empty
                | Node of Order.t * bst * bst

    fun insert (x, Empty) = Node(x, Empty, Empty)
      | insert (x, T as Node(y, left, right)) =
        if Order.lt(x, y) then
            Node(y, insert(x, left), right)
        else if Order.lt(y, x) then
            Node(y, left, insert(x, right))
        else T (* L'elemento esiste già, l'albero resta inalterato *)
end;

(* Step 4: Istanziamento della struttura finale *)
structure StringBST = MakeBST(StringOrder);
```

18.2 Esercizi e approfondimenti dal tutorato (Strutture e BST)

Nelle simulazioni d'esame è frequentissima la richiesta di definire una coda o un dizionario proteggendone lo stato tramite *signature*, oppure di manipolare un BST.

Esercizio: Progettazione ADT della Coda (Queue) Definire una *signature* e un'implementazione opaca per una Coda FIFO.

```
signature QUEUE = sig
  type 'a queue
  exception EmptyQueue
  val emptyQueue: 'a queue
  val enqueue: 'a -> 'a queue -> 'a queue
  val dequeue: 'a queue -> 'a * 'a queue
end;

structure Queue :> QUEUE = struct
  type 'a queue = 'a list
  exception EmptyQueue

  val emptyQueue = []

  (* Inserimento in coda (inefficiente su liste lunghe, ma didattico) *)
  fun enqueue x [] = [x]
    | enqueue x (y::ys) = y :: enqueue x ys

  (* Estrazione dalla testa *)
  fun dequeue [] = raise EmptyQueue
    | dequeue (x::xs) = (x, xs)
end;
```

Esercizio: Estrazione del Minimo in un BST (deletemin) Poiché nel BST i valori più piccoli collassano a sinistra, per trovare e rimuovere il minimo bisogna scendere lungo il ramo sinistro finché non si trova un nodo senza figli a sinistra.

```
exception EmptyTree;

fun deletemin Empty = raise EmptyTree
  (* Caso base: nessun figlio sinistro, questo è il minimo!
   Ritorno il valore e 'aggancio' l'eventuale ramo destro *)
  | deletemin (Node(y, Empty, right)) = (y, right)

  (* Caso ricorsivo: scendo a sinistra *)
  | deletemin (Node(w, left, right)) =
    let
      val (y, new_left) = deletemin(left)
    in
      (y, Node(w, new_left, right))
    end;
```

Esercizio: Ricerca di minimi e massimi in un'unica passata Esercizio avanzato del tutorato che sfrutta le tuple nell'ambiente locale per esplorare simultaneamente i bordi di un albero binario.

```
fun find_min_max Empty = raise EmptyTree
  | find_min_max (Node(v, Empty, Empty)) = (v, v)

(* Ha solo il figlio sinistro *)
| find_min_max (Node(v, left, Empty)) =
  let val (min_sx, max_sx) = find_min_max left
  in (Int.min(v, min_sx), Int.max(v, max_sx)) end

(* Ha solo il figlio destro *)
| find_min_max (Node(v, Empty, right)) =
  let val (min_dx, max_dx) = find_min_max right
  in (Int.min(v, min_dx), Int.max(v, max_dx)) end

(* Ha entrambi i figli *)
| find_min_max (Node(v, left, right)) =
  let
    val (min_sx, max_sx) = find_min_max left
    val (min_dx, max_dx) = find_min_max right
  in
    (Int.min(v, Int.min(min_sx, min_dx)),
     Int.max(v, Int.max(max_sx, max_dx)))
  end;
end;
```

19 Lambda Calcolo

19.1 Spiegazione teorica

Il **Lambda calcolo** (λ -calcolo) è un sistema formale logico-matematico sviluppato da Alonzo Church negli anni '30 per investigare i fondamenti della computazione. Sebbene sia nato decenni prima dei computer moderni, costituisce il cuore teorico e il modello di calcolo di tutti i linguaggi di programmazione funzionali (come LISP, Haskell, ML e Scala).

1. Sintassi del Lambda Calcolo: L'universo del lambda calcolo puro è estremamente essenziale. Non esistono numeri, booleani, stringhe o costrutti di base; esiste solo il concetto di "funzione anonima". Le espressioni (o λ -termini) si definiscono induttivamente tramite tre sole regole:

- **Variabile:** Un identificatore $x, y, z \dots$ è un λ -termine.
- **Astrazione (Creazione di una funzione):** Se e è un λ -termine e x è una variabile, allora $\lambda x.e$ è un λ -termine. Indica la definizione di una funzione anonima il cui parametro formale è x e il cui corpo è e (equivalente a `fn x => e` in ML).
- **Applicazione (Invocazione di una funzione):** Se e_1 ed e_2 sono λ -termini, l'affiancamento $(e_1 e_2)$ è un λ -termine. Rappresenta l'applicazione della funzione e_1 all'argomento e_2 . L'applicazione è associativa a sinistra: $x y z$ equivale a $(x y) z$.

2. Variabili Libere e Legate: In un'astrazione $\lambda x.e$, il simbolo λ funge da operatore di *binding* (legame).

- La variabile x è detta **legata** (bound) all'interno del corpo e .
- Una variabile che compare in un'espressione senza essere vincolata da alcun λ è detta **libera** (free).
- Esempio: In $(\lambda x.x\ y)$, x è legata dal λx , mentre y è libera.

3. Le Regole di Riduzione (La Computazione): Il calcolo progredisce attraverso riscritture sintattiche (riduzioni).

- **Alpha-conversione (α -riduzione):** È il rinominare in modo coerente le variabili legate per evitare "collisioni" o "catture" accidentali di nomi. Esempio: $\lambda x.x$ (la funzione identità) è α -equivalente a $\lambda y.y$ o $\lambda z.z$.
- **Beta-riduzione (β -riduzione):** È il vero motore della computazione. Definisce l'esecuzione di un'applicazione sostituendo il parametro formale con l'argomento attuale.
Sintassi formale: $(\lambda x.e_1)\ e_2 \rightarrow_{\beta} e_1[e_2/x]$
(Si sostituiscono tutte le occorrenze *libere* di x all'interno di e_1 con il termine e_2 , prestando attenzione ad applicare preventivamente un' α -conversione se vi è rischio di sovrapposizioni di nomi).

4. Forma Normale: Un λ -termine si definisce in **forma normale** se non contiene più alcun *redex* (radicale da ridurre). In altre parole, non è più possibile applicare alcuna β -riduzione. La forma normale rappresenta il "risultato" finale del calcolo.

5. Codifiche di Church (Encodings): Poiché nel λ -calcolo puro non ci sono tipi di dato, questi vengono "emulati" strutturalmente usando solo funzioni:

- **Booleani:**
 $Vero \equiv \lambda x.\lambda y.x$ (prende due argomenti e restituisce il primo)
 $Falso \equiv \lambda x.\lambda y.y$ (prende due argomenti e restituisce il secondo)
- **Numeri Naturali (Church Numerals):** I numeri sono codificati come funzioni di ordine superiore che applicano una certa funzione f un numero n di volte a un argomento x .
 $0 \equiv \lambda f.\lambda x.x$
 $1 \equiv \lambda f.\lambda x.f\ x$
 $2 \equiv \lambda f.\lambda x.f\ (f\ x)$

19.2 Implementazione e Riduzioni Logiche

Vediamo alcuni esempi classici di β -riduzione.

19.2.1 Esempio 1: Applicazione di Base

Valutazione di un'applicazione semplice:

Termine: $(\lambda x.(\lambda z.x\ z))\ y$

Regola: Il parametro formale è x , l'argomento è y .

Sostituiamo tutte le occorrenze di x nel corpo $(\lambda z.x\ z)$ con y .

Risultato: $\lambda z.y\ z$

(Il risultato è in forma normale in quanto non ci sono altre applicazioni).

19.2.2 Esempio 2: Valutazione di un Booleano con operatore

Modelliamo il costrutto `if True then A else B`:

Definizione If: $\backslash b.\backslash x.\backslash y. b \ x \ y$

Definizione True: $\backslash u.\backslash v. u$

Valutazione di $(\text{If True } A \ B)$:

1. $(\backslash b.\backslash x.\backslash y. b \ x \ y) (\backslash u.\backslash v. u) \ A \ B$
2. Sostituiamo b : $(\backslash x.\backslash y. (\backslash u.\backslash v. u) \ x \ y) \ A \ B$
3. Sostituiamo x con A : $(\backslash y. (\backslash u.\backslash v. u) \ A \ y) \ B$
4. Sostituiamo y con B : $(\backslash u.\backslash v. u) \ A \ B$
5. Sostituiamo u con A : $(\backslash v. A) \ B$
6. Sostituiamo v con B : A

Risultato corretto: Il ramo "True" ha restituito A .

19.3 Esercizi e approfondimenti dal tutorato: Computazione in Forma Normale

All'esame viene spesso fornito un lungo e complesso termine di Lambda Calcolo e viene richiesto di mostrarne passo-passo la riduzione fino all'ottenimento della Forma Normale. L'accortezza fondamentale risiede nell'evitare le collisioni applicando correttamente le parentesi e la precedenza (associativa a sinistra).

19.3.1 Esercizio L11.12: Riduzione annidata

Si riduca il seguente termine a forma normale:

$(\backslash x. \ x \ z \ x) ((\backslash y. \ y \ y \ x) \ z)$

Svolgimento Passo-Passo:

1. Notiamo che c'è un'applicazione annidata sulla destra. Conviene prima ridurre l'argomento (*Applicative Order*):

Sotto-termine destro: $(\backslash y. \ y \ y \ x) \ z$

Sostituisco y con z : $z \ z \ x$

2. Il nostro termine principale è ora semplificato:

$(\backslash x. \ x \ z \ x) (z \ z \ x)$

3. Attenzione alle collisioni! C'è un rischio di cattura di variabile per la x finale. Per procedere in assoluta pulizia formale (anche se qui le x combaciano come variabili libere di contesto), sostituiamo l'intero blocco (zzx) al posto delle x legate dal primo λ :

$(z \ z \ x) \ z \ (z \ z \ x)$

Risultato Finale: $(zzx)z(zzx)$. L'espressione non contiene più alcun simbolo di astrazione λ su cui effettuare applicazioni, pertanto è in forma normale.

19.3.2 Esercizio L11.13: Riduzione con Catture Evitate

Si riduca il seguente termine a forma normale:

Termine Iniziale:

$(\lambda x. x y) (\lambda t. t z) ((\lambda x. \lambda z. x y z) y x)$

Svolgimento Passo-Passo:

1. Isoliamo il blocco destro e risolviamolo per semplificare il lavoro. Notiamo l'applicazione di due argomenti in cascata (y e poi x).

Blocco destro: $((\lambda x. \lambda z. x y z) y) x$

Applico y alla x : $(\lambda z. y y z) x$

Applico x alla z : $y y x$

2. Il termine intero diventa ora un'applicazione sequenziale:

$(\lambda x. x y) (\lambda t. t z) (y y x)$

3. Il Lambda calcolo è associativo a sinistra. Appliciamo quindi il primo termine al secondo:

Sotto-termine sinistro: $(\lambda x. x y) (\lambda t. t z)$

Sostituisco $(\lambda t. t z)$ al posto di x : $(\lambda t. t z) y$

Applico y al posto di t : $y z$

4. Reinseriamo il risultato nell'equazione principale, unendo i due blocchi:

$(y z) (y y x)$

Risultato Finale: $(yz)(yyx)$. Tutte le funzioni astratte sono state risolte.

20 Programmazione Logica e Prolog

20.1 Spiegazione teorica

Il paradigma logico si discosta radicalmente sia dalla programmazione imperativa (che descrive *come* eseguire un calcolo modificando lo stato) sia da quella funzionale (che valuta espressioni e funzioni). La programmazione logica è ****dichiarativa****: il programmatore descrive la struttura logica del problema (le relazioni tra le entità) e il motore interno del linguaggio si occupa di trovare la soluzione.

Fondamenti del Paradigma Logico:

- **Relazioni vs Funzioni:** In matematica, una funzione $f(x) = y$ mappa un input a un output. Una relazione $R(x, y)$ afferma semplicemente che x e y sono legati in qualche modo. La programmazione logica si basa sulle relazioni.
- **Prolog (Programming in Logic):** È il linguaggio di programmazione logica più celebre, basato su un sottoinsieme della logica del primo ordine (le *Clause di Horn*).

Sintassi base di Prolog:

- **Atomi:** Costanti, nomi, o simboli. Iniziano rigorosamente con una lettera ****minuscola**** (es. `john`, `rochester`, `a`).
- **Variabili:** Iniziano rigorosamente con una lettera ****maiuscola**** o con un underscore (es. `X`, `Person`, `_`). L'underscore singolo `_` è la variabile anonima (corrisponde alla wildcard in ML).
- **Fatti (Facts):** Asserzioni incondizionate considerate sempre vere nel nostro "mondo" o database. Es: `padre(john, mary)`.
- **Regole (Rules):** Asserzioni condizionate (implicazioni logiche). Hanno la forma `Testa :- Corpo`. Il simbolo `:-` si legge "se" (if). Es: `nonno(X, Z) :- padre(X, Y), padre(Y, Z)`. La virgola `,` rappresenta l'AND logico, mentre il punto e virgola `;` rappresenta l'OR.
- **Query / Goal:** Interrogazioni poste al sistema (precedute dal prompt `?-`). Prolog risponde **true** (o **yes**) se la query è deducibile, **false** (o **no**) altrimenti, oppure restituisce le associazioni (binding) delle variabili che rendono vera la query.

Unificazione e Risoluzione:

- L'**Unificazione** è il processo mediante il quale Prolog cerca di rendere identici due termini assegnando valori alle variabili. Ad esempio, per unificare `padre(john, X)` e `padre(Y, mary)`, Prolog istanzia `Y = john` e `X = mary`.
- Il motore di **Risoluzione** esplora il database dall'alto verso il basso (top-down) e risolve le regole da sinistra verso destra (left-to-right).
- **Backtracking (Ritorsione):** Se Prolog intraprende un percorso di ricerca che fallisce (arriva a un punto cieco in cui non può unificare), "torna indietro" all'ultimo punto in cui c'era un'alternativa (choice point) e tenta una strada diversa.

Operatori Speciali:

- **Is (Aritmetica):** In Prolog, `X = 3 + 4` non calcola 7, ma unifica la variabile `X` con la struttura sintattica `3 + 4`. Per forzare la valutazione aritmetica si usa l'operatore `is`: `X is 3 + 4`.
- **Cut (!) (Taglio):** È un operatore extra-logico che blocca il backtracking. Consente di ottimizzare la ricerca ("potatura dell'albero") impedendo a Prolog di esplorare alternative che sappiamo essere inutili o errate.

- **Negation as Failure ($\backslash +$ o **not**):** In Prolog, se un fatto non può essere dimostrato, viene considerato falso (Assunzione del Mondo Chiuso). **not(P)** è vero se e solo se l'esecuzione di P fallisce.

Liste in Prolog: Strutturalmente identiche a quelle in ML. Si indicano con le parentesi quadre []. La sintassi per destrutturare una lista in Testa e Coda usa il pipe |: [Head | Tail].

20.2 Implementazione del codice

20.2.1 Liste, Ricorsione e Backtracking (append e member)

A differenza dei linguaggi funzionali in cui le funzioni hanno input e output definiti, in Prolog una singola definizione relazionale può operare in molteplici direzioni (invertibilità).

```
% Definizione della relazione "Membro di una lista"
% 1. Caso base: X è membro della lista se è la testa della lista.
member(X, [X | _]).

% 2. Passo ricorsivo: X è membro se è membro della coda della lista.
member(X, [_ | Tail]) :- member(X, Tail).

% Definizione dell'Append (concatenazione di liste)
% append(L1, L2, L3) è vero se L3 è la concatenazione di L1 e L2.
append([], L, L).
append([H | T1], L2, [H | T3]) :- append(T1, L2, T3).

% POTENZA DICHIARATIVA:
% Interrogazione standard: ?- append([a,b], [c,d], X).
%   -> X = [a,b,c,d]
% Sottrazione di liste:    ?- append(X, [c,d], [a,b,c,d]).
%   -> X = [a,b]
% Generazione di pattern: ?- append(X, Y, [a,b,c]).
%   -> X=[], Y=[a,b,c] ; X=[a], Y=[b,c] ; ...
```

20.2.2 Tic-Tac-Toe in Prolog (Tris Interattivo)

Il file `tic-tac-toe-interactive.pl` fornito a lezione illustra come unire le regole dichiarative per la vittoria, il backtracking per la scelta della mossa e l'I/O (Input/Output).

```
% 1. Dichiariamo che i predicati 'o' (computer) e 'x' (giocatore)
% sono dinamici (il database può essere modificato a runtime)
:- dynamic o/1.
:- dynamic x/1.

% 2. Definizione delle condizioni di vittoria (Linee vincenti)
ordered_line(1, 2, 3).
ordered_line(4, 5, 6).
ordered_line(7, 8, 9).
ordered_line(1, 4, 7).
% ... altre combinazioni diagonali ...

% Permutazioni per controllare la linea in qualsiasi ordine
line(A, B, C) :- ordered_line(A, B, C).
line(A, B, C) :- ordered_line(A, C, B).
% ... (altre permutazioni delegate dal Cut o dal Backtracking)

% 3. Logica di intelligenza artificiale: "Cosa è una buona mossa?"
% Prolog tenta le regole in ordine (dall'alto verso il basso).
move(A) :- good(A), empty(A).

% Condizione 1: Cerca di vincere
good(A) :- win(A).
% Condizione 2: Blocca la vittoria del giocatore
good(A) :- block_win(A).
% Condizioni fallback: occupa il centro, poi gli angoli
good(5).
good(1).
good(3).

% Implementazione di 'win' e 'block_win'
% "Vinciamo se abbiamo due 'o' su una linea e A è la terza casella vuota"
win(A) :- o(B), o(C), line(A, B, C).
% "Blocchiamo se l'utente 'x' ha due caselle su una linea"
block_win(A) :- x(B), x(C), line(A, B, C).

% 4. Gestione dello stato (Effetti collaterali con asserta)
makemove :- move(X), !, asserta(x(X)). % Il Cut (!) ferma la ricerca
                                         % appena trova una mossa valida
makemove :- all_full.                  % Fallback in caso di pareggio
```

Commento Architetture per l'esame: L'uso di `:- dynamic` abbinato al comando `asserta` (che inserisce dinamicamente un nuovo fatto nel database all'inizio) e `retract` (che lo elimina) permette a Prolog di mantenere uno "stato" mutevole, aggirando la pura immutabilità funzionale. Inoltre, il cut `!` in `makemove` è fondamentale: senza di esso, se la mossa successiva portasse a un fallimento o al backtracking, Prolog annullerebbe l'inserimento della mossa per provare il `good` successivo, generando comportamenti indesiderati nell'albero di gioco.

21 Simulazioni d'Esame: Preparazione Teorica

Questa sezione raccoglie in modo strutturato e ragionato una vasta casistica di domande teoriche e a risposta multipla provenienti dagli appelli passati e dalle simulazioni ufficiali. È pensata per essere un banco di prova completo prima dell'esame, diviso per macro-aree tematiche.

21.1 Ambiente, Regole di Scope e Passaggio dei Parametri

- **Domanda 1**

Testo: L'ambiente (o environment) è:

Risposta esatta: L'insieme delle associazioni (nome, entità denotabile) esistenti in uno specifico punto del programma ed in uno specifico momento durante l'esecuzione di un programma.

Spiegazione: Da non confondere con la "memoria" (che associa locazioni a valori). L'ambiente è una mappa logica dinamica che collega gli identificatori (variabili, funzioni, tipi) scelti dal programmatore alle rispettive entità semantiche.

- **Domanda 2**

Testo: Quale/i delle seguenti affermazioni sugli oggetti denotabili è vera?

Risposta esatta: Un oggetto denotabile è un oggetto a cui può essere associato un nome (ad es. i nomi delle variabili definiti dagli utenti).

Spiegazione: Qualsiasi costrutto a cui il linguaggio permette di assegnare un identificatore (una costante in ML, una funzione, una classe in Scala) è denotabile.

- **Domanda 3**

Testo: L'ambiente non locale di un blocco di codice è:

Risposta esatta: L'insieme dei binding visibili dentro al blocco, ma non direttamente definiti in esso.

Spiegazione: Se una funzione richiama una variabile globale o una variabile definita in una funzione esterna che la avvolge, sta accedendo al suo ambiente non-locale.

- **Domanda 4**

Testo: Dato il seguente pseudocodice:

```
int y = 12;
int f1(int x) { x = 12; return x / 2; }
int f2(void) {
    int a, c;
    a = y / 3;
    c = f1(a);
    return a + c;
}
```

Qual è il valore di ritorno di `f2` se i parametri sono passati **per valore**?

Risposta esatta: 10

Spiegazione: In `f2`, si calcola $a = 12 / 3 = 4$. Chiamando `f1(a)`, viene passata una copia del valore 4. All'interno di `f1`, la variabile locale `x` sovrascrive il 4 con 12 e ritorna $12 / 2 = 6$. Dunque $c = 6$. Poiché il passaggio è avvenuto per valore, la `a` originale in `f2` è rimasta inalterata a 4. Il ritorno finale è $4 + 6 = 10$.

- **Domanda 5**

Testo: Dato il seguente pseudocodice:

```
int x = 1; int y = 2;
void cambia() { x = x + y; }
void bloccoA() { int y = 10; cambia(); }
```

Se lo scoping è **dinamico**, quanto vale `x` dopo l'esecuzione di `bloccoA()`?

Risposta esatta: 11

Spiegazione: Lo scoping dinamico cerca il valore della variabile nell'ultimo record di attivazione impilato sullo Stack, risalendo la catena dinamica delle chiamate. Poiché `cambia()` è invocato da `bloccoA()`, esso non vedrà la `y` globale (2), ma la `y` locale di `bloccoA` (10). La variabile `x` globale viene quindi aggiornata a $1 + 10 = 11$.

- **Domanda 6**

Testo: Quali delle seguenti affermazioni sulle binding policy sono vere?

Risposta esatta: Nello shallow binding l'environment da considerare è quello attivo al momento della chiamata della funzione passata come parametro.

Spiegazione: Questo si contrappone al *Deep Binding* (tipico di ML in congiunzione con le *Closures*), in cui l'ambiente viene "congelato" ed associato alla funzione nel momento in cui essa viene definita, a garanzia della sicurezza dei tipi e dello stato.

21.2 Memoria, Macchine Astratte e Garbage Collection

- **Domanda 7**

Testo: I puntatori di catena dinamica (Dynamic Link) contenuti in un record di attivazione:

Risposta esatta: Permettono, a partire da un RdA, di trovare il RdA precedente sullo stack (quello della procedura chiamante).

Spiegazione: Formano la catena delle chiamate a funzione a runtime, essenziali sia per lo smontaggio (pop) corretto dello stack alla terminazione di una funzione, sia per la ricerca di variabili nello scoping dinamico.

- **Domanda 8**

Testo: La frammentazione **interna** causa:

Risposta esatta: Uno spreco di memoria.

Spiegazione: Si verifica quando il sistema operativo assegna a un processo un blocco di memoria leggermente più grande di quello richiesto; la parte inutilizzata all'interno del blocco assegnato non può essere usata da nessun altro.

- **Domanda 9**

Testo: La frammentazione **esterna** causa:

Risposta esatta: L'impossibilità di allocare grandi blocchi di memoria anche se la memoria libera totale è sufficiente.

Spiegazione: Avviene quando la memoria Heap libera è spezzettata in tanti piccoli frammenti non contigui, rendendo fallimentare una richiesta per un singolo blocco massiccio di memoria (es. un grosso array).

- **Domanda 10**

Testo: Un Garbage Collector:

Risposta esatta: Gestisce in maniera automatica la deallocazione di oggetti non più utilizzati dal programma, evitando memory leak. Non è esclusiva dei linguaggi funzionali ed esistono varie tecniche (es. Mark and Sweep).

Spiegazione: Il suo compito primario è scandagliare la memoria Heap alla ricerca di oggetti il cui puntatore/riferimento non è più raggiungibile dallo Stack, deallocandoli in sicurezza (es. Java).

- **Domanda 11**

Testo: Si può dire che una macchina astratta che capisce il linguaggio Java non sia implementata in modo puramente compilativo perché:

Risposta esatta: La macchina virtuale di Java (JVM) o JRE deve comunque essere presente per interpretare il Bytecode.

Spiegazione: Java utilizza un'architettura mista. Il compilatore (`javac`) non produce codice macchina nativo, ma Bytecode. È poi l'interprete (la JVM) a convertire istantaneamente il Bytecode in istruzioni macchina durante l'esecuzione (JIT compilation).

21.3 Lambda Calcolo e Codifiche

- **Domanda 12**

Testo: Nella sostituzione $(\lambda a.abc)[d/c]$:

Risposta esatta: Non c'è alcuna cattura di variabile.

Spiegazione: Sostituiamo la c (libera) con d . Otteniamo $\lambda a.abd$. Non c'è collisione o cattura perché d non entra nel raggio di un legame locale pre-esistente con lo stesso nome. Diverso sarebbe stato se avessimo voluto sostituire c con a ; in tal caso a sarebbe stata "catturata" dal λa , e avremmo dovuto usare un' α -conversione (es. $\lambda z.zbc$) prima.

- **Domanda 13**

Testo: Beta-riducendo $(\lambda a.aaa)(\lambda b.b)(\lambda c.c)$ si ottiene:

Risposta esatta: $\lambda c.c$

Spiegazione: 1. Si applica il primo blocco al secondo. Tutte le a diventano la funzione identità $(\lambda b.b)$.

2. Otteniamo: $(\lambda b.b)(\lambda b.b)(\lambda b.b)$ e a parte il nostro terzo termine originale $(\lambda c.c)$.

3. Riducendo la catena di identità: $(\lambda b.b)$ applicata a se stessa ridà $(\lambda b.b)$. Questa, riapplicata ancora, dà di nuovo $(\lambda b.b)$.

4. Infine, assemblando l'ultimo pezzo: $(\lambda b.b)(\lambda c.c)$. L'identità applicata al termine $\lambda c.c$ ci restituisce intatto $\lambda c.c$.

- **Domanda 14**

Testo: Beta-riducendo $(\lambda n.\lambda f.\lambda x.f((nf)x))(\lambda f.\lambda x.f(f(fx)))$ si ottiene:

Risposta esatta: $\lambda f.\lambda x.f(f(f(ffx)))$

Spiegazione: Il primo termine è la funzione *SUCC* (Successore) nella codifica dei Church Numerals. Il secondo termine è il numero 3 di Church (la funzione f applicata 3 volte). Applicando il Successore al numero 3, si ottiene logicamente (e sintatticamente) il numero 4 di Church, cioè la f applicata 4 volte alla x .

- **Domanda 15**

Testo: Beta-riducendo $(\lambda a.((a\lambda b.\lambda c.c)\lambda d.\lambda e.d))(\lambda f.\lambda g.f)$ si ottiene:

Risposta esatta: $\lambda b.\lambda c.c$

Spiegazione: Questa è la codifica condizionale Booleana (**IF-THEN-ELSE**). Il termine $(\lambda f.\lambda g.f)$ è il "TRUE" di Church (prende due argomenti e restituisce il primo). Sostituendo il True al posto della a nel blocco centrale, l'espressione dice: **TRUE (False) (True)**. Poiché è True, la funzione scarta il secondo argomento e restituisce il primo, che è la funzione $(\lambda b.\lambda c.c)$ (il False di Church).

- **Domanda 16**

Testo: Data la lambda-espressione $\lambda a.(\lambda b.bf(\lambda c.dcag))$, quali sono le variabili *free* (libere)?

Risposta esatta: f, d, g

Spiegazione: Le variabili a, b, c sono strettamente vincolate ("bound") dalle relative astrazioni (rispettivamente $\lambda a, \lambda b, \lambda c$). Le lettere f, d, g appaiono nel corpo ma non possiedono alcun operatore λ che le lega nel contesto dell'espressione fornita, rendendole libere.

21.4 Sistemi di Tipi, Polimorfismo e ML

- **Domanda 17**

Testo: In ML, il costrutto `>` :

Risposta esatta: È usato per nascondere le componenti di una struttura che non sono visibili nella signature (Ascription opaca).

Spiegazione: Rappresenta l'implementazione pratica dell'**Abstract Data Type (ADT)**. Il tipo e le definizioni vengono sigillati, garantendo che gli utenti esterni possano interagire con i dati solo attraverso le funzioni esposte nella signature, garantendone la consistenza.

- **Domanda 18**

Testo: Qual è il tipo dell'espressione `fn a => fn b => fn c => (a b) / c ?`

Risposta esatta: `fn: ('a -> real) -> 'a -> real -> real`

Spiegazione: Procediamo per vincoli (*Type Inference*): l'operatore `/` è la divisione in virgola mobile (esclusiva dei reali in ML). Questo forza il divisore `c` a essere `real`, e forza il risultato del blocco `(a b)` a essere `real`. Di conseguenza: `b` ha un tipo generico `'a`, mentre `a` deve essere una funzione applicabile a `b` per produrre un reale, quindi `('a -> real)`.

- **Domanda 19**

Testo: Qual è il tipo della seguente espressione: `fun f x y = x y; ?`

Risposta esatta: `fn: ('a -> 'b) -> 'a -> 'b`

Spiegazione: La notazione per affiancamento `x y` indica l'applicazione della funzione `x` all'argomento `y`. Se diamo a `y` il tipo generico `'a`, `x` dovrà essere una funzione che assorbe `'a` ed emette `'b`. Il risultato totale della funzione è l'output di `x`.

- **Domanda 20**

Testo: Siano dati i tipi riferimento `A` e `B`, e i tipi generici `C<A>` e `C<B>`. Se `B` è una sottoclasse di `A`, allora `C<B>` è una sottoclasse di `C<A>`.

Risposta esatta: Falso

Spiegazione: I tipi generici (sia in Java che in altri ecosistemi tipizzati visti) sono **invarianti**. Una Lista di Interi non eredita e non può essere trattata direttamente come una Lista di Object (se lo fosse, si potrebbero inserire stringhe in una lista di interi sfondando la Type Safety a runtime).

- **Domanda 21**

Testo: Nel seguente programma ML:

```
fun pow x 0 = 1.0
  | pow x y = x * pow x (y - 1);
```

Il tipo dell'espressione `pow 5.0` è:

Risposta esatta: `fn: int -> real`

Spiegazione: È un classico caso di applicazione parziale (*Currying*). Il tipo originale della funzione è `real -> int -> real`. Fornendo unicamente la base (5.0), l'ambiente "scongela" e assorbe il primo argomento, restituendo una nuova funzione in attesa del solo esponente di tipo `int`.

22 Simulazioni d'Esame: Esercizi Pratici in ML

In questa sezione finale raccogliamo i problemi di programmazione pratica in linguaggio Standard ML proposti durante le simulazioni ufficiali e gli appelli passati. In sede d'esame, la valutazione di questi esercizi si basa sulla corretta applicazione del paradigma funzionale: divieto assoluto di usare costrutti imperativi (come i cicli o i riferimenti mutabili), uso elegante del *Pattern Matching*, corretta inferenza dei tipi e gestione efficiente dell'ambiente locale tramite i blocchi `let ... in ... end`.

22.1 Esercizio 1: Estrazione dei percorsi su Alberi (Appello 15/07/2025)

Testo del problema:

Dato il seguente tipo di dato che rappresenta un albero di numeri interi:

```
datatype Tree = Leaf of int | Node of int * Tree * Tree | Empty;
```

Scrivere due funzioni. La prima, `all_paths (Tree -> int list list)`, deve restituire tutti i percorsi dalla radice dell'albero alle foglie (`Empty` non è considerato una foglia). La seconda, `filter_paths ('a list * ('a -> bool) -> 'a list)`, prende in input una lista di path e un predicato generico, e restituisce una lista di liste contenente solo i path che soddisfano tale condizione. *È ammesso l'utilizzo della funzione `map` ma non di altre funzioni di libreria.*

Soluzione (Valutata 30L):

```
datatype Tree = Leaf of int | Node of int * Tree * Tree | Empty;
```

```
fun all_paths(t: Tree) =
```

```
let
```

```
    fun aux (Empty) = []
```

```
      | aux (Leaf n) = [[n]]
```

```
      | aux (Node (n, l, r)) =
```

```
        let
```

```
            val left_paths = aux(l)
```

```
            val right_paths = aux(r)
```

```
        in
```

```
            (* Aggiunge il nodo corrente 'n' in testa a
```

```
              TUTTI i percorsi calcolati dai figli *)
```

```
            map (fn path => n::path) (left_paths @ right_paths)
```

```
        end
```

```
in
```

```
    aux(t)
```

```
end;
```

```
fun filter_paths ([], f) = []
```

```
  | filter_paths (x::xs, f) =
```

```
    if (f(x)) then x :: filter_paths(xs, f)
```

```
    else filter_paths(xs, f);
```

```
(* Esempio di utilizzo (filtro basato sulla somma degli elementi): *)
```

```
fun sum_lower_than_30 list = (foldl (op +) 0 list) < 30;
```

Analisi didattica: L'utilizzo del `map` con una funzione anonima (`fn path => n::path`) per concatenare il nodo radice a *tutte* le liste figlie in un colpo solo dimostra un'ottima padronanza delle funzioni di ordine superiore. Il blocco `let` evita di ricalcolare i percorsi più di una volta.

22.2 Esercizio 2: Astrazione del Controllo (Il ciclo FOR)

Testo del problema (Prog 3):

Si consideri il tipo di dato `datatype FOR = For of int * (int -> int)`, i cui valori `For(n, f)` rappresentano funzioni che implementano un ciclo `for` imperativo che applica iterativamente n volte la funzione f all'argomento. Si scriva una funzione `eval` (avente tipo `FOR -> (int -> int)`) che riceve come argomento un valore di tipo `FOR` e ritorna una funzione `curried` pronta ad applicare tale ciclo. Se f moltiplica per 2, `eval (For(3, f))` ritornerà una funzione che dato un input lo moltiplicherà per 8 (2^3).

Soluzione:

```
datatype FOR = For of int * (int -> int);

(* Caso base: un ciclo che itera 0 volte restituisce la funzione Identità *)
fun eval (For (0, f)) = (fn x => x)
  | eval (For (n, f)) =
    let
      (* Scongela il passo successivo (n-1) *)
      val rest_eval = eval(For(n - 1, f))
    in
      (* Restituisce una closure che compone f con il resto delle
         iterazioni *)
      (fn x => f(rest_eval(x)))
    end;

(* Alternativa sintatticamente perfetta usando la composizione 'o': *)
fun eval2 (For (0, f)) = (fn x => x)
  | eval2 (For (n, f)) = f o eval2(For (n - 1, f));
```

Analisi didattica: Questo esercizio testa la comprensione profonda della *Partial Instantiation* e delle *Closure*. La funzione `eval` non calcola il risultato finale matematico, ma si limita ad "assemblare" la catena di funzioni $f(f(f(x)))$ e la restituisce in attesa che l'utente fornisca l'input x .

22.3 Esercizio 3: Somma Filtrata per Parità (Prog 1)

Testo del problema:

Si scriva una funzione `sommali` (avente tipo `int -> int list -> int`) che riceve un intero n ed una lista di interi L . La funzione somma ad n gli elementi di L che hanno posizione pari (indici 0, 2, 4...). Se la lista contiene meno di 2 elementi, la logica si applica regolarmente. (Es: `sommali 2 [1,2,3,4] = 2 + 1 + 3 = 6`).

Soluzione:

```
fun sommali n [] = n
  | sommali n [x] = n + x
  (* De-strutturiamo la lista afferrando i primi due elementi:
     x (pari) e y (dispari). Sommiamo x al totalizzatore, e scartiamo y. *)
  | sommali n (x::y::xs) = sommali (n + x) xs;
```

Analisi didattica: Raggiunge un'efficienza ottimale sfruttando la **Tail Recursion** (Ricorsione in coda) in modo che l'accumulatore logico venga trascinato nelle chiamate successive, non riempiendo mai il *Call Stack*. L'uso del pattern `x::y::xs` permette di "saltare" le posizioni dispari con una sintassi compatta.

22.4 Esercizio 4: Manipolazione di stringhe (Prefissi e Suffissi)

Testo del problema (Exam Examples):

Scrivere due funzioni `suffixes` e `prefixes` (tipo `string -> string list`) che, data una stringa, restituiscano la lista di tutti i suoi suffissi/prefissi, ignorando la stringa vuota. Esempio: `suffixes "ciao" = ["ciao", "iao", "ao", "o"]`.

Soluzione:

```
(* SUFFISSI: Semplice, poiché possiamo usare l'estrazione substring *)
fun suffixes "" = []
  | suffixes str =
    str :: suffixes (String.extract(str, 1, NONE));

(* PREFISSI: Convertiamo in char list per poter comporre gli elementi *)
fun pref_aux [] = []
  | pref_aux (x::xs) =
    (* Il primo carattere si abbina a tutti i prefissi generati in coda *)
    [x] :: map (fn p => x::p) (pref_aux xs);

fun prefixes "" = []
  | prefixes str =
    (* implode trasforma le char list interne di nuovo in stringhe *)
    map implode (pref_aux(explode str));
```

22.5 Esercizio 5: Addizione Binaria Nativa su Liste

Testo del problema (Exam Examples):

Scrivere la funzione `sum_binary` di tipo `int list * int list -> int list` che prende in input una coppia di liste di interi (composte da 1 e 0) rappresentanti un numero binario, e ne restituisca la somma binaria. Esempio: `sum_binary ([1,0], [1,0]) = [1,0,0]`.

Soluzione:

```
(* Funzione ausiliaria che incapsula la logica del riporto (carry) *)
fun sum_bin_carry ([], [], 0) = []
  | sum_bin_carry ([], [], 1) = [1]

(* Se una delle due liste è più corta, aggiungiamo [0] fittizi *)
| sum_bin_carry (x::xs, [], carry) = sum_bin_carry(x::xs, [0], carry)
| sum_bin_carry ([], y::ys, carry) = sum_bin_carry([0], y::ys, carry)

(* Passo generale per l'elaborazione dei bit *)
| sum_bin_carry (x::xs, y::ys, carry) =
  let
    val sum = x + y + carry
    val new_bit = sum mod 2    (* Se la somma è 2, il bit è 0 *)
    val new_carry = sum div 2 (* Se la somma è 2 o 3, il carry è 1 *)
  in
    new_bit :: sum_bin_carry(xs, ys, new_carry)
  end;

(* Wrapper esterno che inverte le liste (Little-Endian vs Big-Endian) *)
fun sum_binary (L1, L2) =
  rev (sum_bin_carry (rev L1, rev L2, 0));
```

Analisi didattica: In ML, l'operatore `cons (::)` permette di aggiungere elementi solo *in testa* alla lista, operazione logica $O(1)$. Poiché un'addizione richiede l'allineamento a destra (sui bit meno significativi), è obbligatorio ribaltare prima gli operandi con la funzione nativa `rev`, processare i carry in avanti e, infine, ribaltare il risultato. L'uso dei pattern per gestire la discrepanza di lunghezza tra gli operandi è un'altra peculiarità vitale del codice.

22.6 Esercizio 6: Aggregazione su Strutture Ramificate (get_words)

Testo del problema (Exam Examples):

Dato il tipo datatype `ctree = Empty | Leaf of char | Node of char * ctree * ctree`. Scrivere una funzione `get_words` che restituisce una coppia di stringhe (`string * string`). La prima stringa è la concatenazione dei caratteri contenuti nelle foglie, la seconda stringa è la concatenazione di quelli nei nodi interni. (Es: `get_words Node("#a", Leaf #"b", Leaf #"c") = ("bc", "a")`).

Soluzione:

```
datatype ctree = Empty | Leaf of char | Node of char * ctree * ctree;

fun get_words Empty = ("", "")
  | get_words (Leaf c) = (str(c), "")
  | get_words (Node (c, left, right)) =
    let
      (* Estraiamo i risultati pre-calcolati dai sotto-alberi *)
      val (leavesL, nodesL) = get_words(left)
      val (leavesR, nodesR) = get_words(right)
    in
      (* Ricomponiamo la tupla assemblando i rispettivi domini *)
      (leavesL ^ leavesR, str(c) ^ nodesL ^ nodesR)
    end;
```

Analisi didattica: Questa soluzione sfrutta l'ambiente locale `let` in congiunzione con l'operatore nativo di concatenazione di stringhe (`^`) e il convertitore `str()` per operare il casting da carattere a stringa, aggirando costose manipolazioni di liste di caratteri intermedie.